

dispatch_calibration_validation_calibrated_demand

May 20, 2026

[]:

1 Module 13n and Module 14c Ablation Dispatch Validation

Standalone comparison report for the South Africa 2023 fixed-fleet dispatch calibration after Module 13n, with the removable Module 14c ablation sequence overlaid as model cases. The notebook keeps the same tables, plots, and diagnostic sections as the main dispatch calibration notebook, but compares Eskom CY2023 historical operations against:

- LOW-GAS-OFFICIAL-FLEET — accepted Module 13n calibrated-demand baseline.
- MODULE14C-COAL485-NUCLEAR50-NO-VRE-NO-OCGT-CAP — coal EAF 0.485 + nuclear p_max_pu = 0.5, no VRE scaling, no OCGT annual cap.
- MODULE14C-COAL485-NUCLEAR50-VRE-ONLY — same coal/nuclear correction plus Module 14 VRE scaling, no OCGT annual cap.
- MODULE14C-COAL485-NUCLEAR50 — same coal/nuclear correction plus VRE scaling and the 5.243 TWh OCGT cap.
- MODULE14B-COAL49-NUCLEAR50 — coal 49% comparator retained to show the scarcity tradeoff.

1.1 0. Setup and data loading

Load the calibrated-demand official-fleet solve and Eskom 2023 reference data, then convert the network into aligned hourly carrier time series. The scenario registry intentionally contains one model case only: the Module 13n LOW-GAS-OFFICIAL-FLEET network. Carrier dispatch is compared to Eskom historical carrier series; the demand/accounting rows use the Module 13n calibrated historical demand series rather than raw RSA Contracted Demand.

```
[111]: from pathlib import Path
import logging
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Markdown
from scipy.stats import ConstantInputWarning, pearsonr, spearmanr

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```

warnings.filterwarnings("ignore", category=RuntimeWarning)
warnings.filterwarnings("ignore", category=ConstantInputWarning)
logging.getLogger("matplotlib.category").setLevel(logging.ERROR)
logging.getLogger("pypsa").setLevel(logging.WARNING)
logging.getLogger("pypsa.io").setLevel(logging.WARNING)

try:
    import pypsa
except Exception as exc:
    pypsa = None
    print(f"WARNING: PyPSA unavailable: {exc}")

plt.rcParams.update({
    "figure.dpi": 120,
    "savefig.dpi": 140,
    "font.size": 9,
    "axes.titlesize": 11,
    "axes.labelsize": 9,
    "legend.fontsize": 8,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "axes.grid": True,
    "grid.alpha": 0.25,
    "grid.linewidth": 0.6,
})

ROOT = next((candidate for candidate in [Path.cwd(), *Path.cwd().parents] if
    ↪(candidate / "Snakefile").exists()), Path.cwd())
RUN = "za_2023_fixed_validation"
RESULTS = ROOT / "results" / RUN / "networks"
DATA_VALIDATION = ROOT / "data" / "za_validation"
DATA_AUDIT = ROOT / "data" / "za_audit"

COAL_P_NOM_TOLERANCE_MW = 2.0

# Scenario registry for the calibrated-demand comparison.
# Module 13n is the accepted baseline. Module 14c ablations isolate the
# coal/nuclear correction, VRE scaling, and OCGT annual cap.
SCENARIOS = {
    "low_gas_official_fleet": {
        "label": "LOW-GAS-OFFICIAL-FLEET",
        "paths": [
            RESULTS / ↪
            ↪"elec_s_34_ec_lc1_NoCO2-1H-EAF-UC-OPC-LOW-GAS-OFFICIAL-FLEET.nc",
            RESULTS / "elec_s_34_ec_lc1_NoCO2-1H-EAF-UC-OPC-LOW-GAS.nc",
        ],
        "coal_p_nom_mw": 39099.0,
    }
}

```

```

    },
    "module14c_coal485_nuclear50_no_vre_no_ocgt_cap": {
        "label": "MODULE14C-COAL485-NUCLEAR50-NO-VRE-NO-OCGT-CAP",
        "paths": [
            RESULTS /
↪"elec_s_34_ec_lc1_NoCO2-1H-EAF-UC-OPC-LOW-GAS-OFFICIAL-FLEET-MODULE14C-COAL485-NUCLEAR50-NO-
↪nc",
        ],
        "coal_p_nom_mw": 39099.0,
    },
    "module14c_coal485_nuclear50_vre_only": {
        "label": "MODULE14C-COAL485-NUCLEAR50-VRE-ONLY",
        "paths": [
            RESULTS /
↪"elec_s_34_ec_lc1_NoCO2-1H-EAF-UC-OPC-LOW-GAS-OFFICIAL-FLEET-MODULE14C-COAL485-NUCLEAR50-VRE-
↪nc",
        ],
        "coal_p_nom_mw": 39099.0,
    },
    "module14c_coal485_nuclear50": {
        "label": "MODULE14C-COAL485-NUCLEAR50",
        "paths": [
            RESULTS /
↪"elec_s_34_ec_lc1_NoCO2-1H-EAF-UC-OPC-LOW-GAS-OFFICIAL-FLEET-MODULE14C-COAL485-NUCLEAR50.
↪nc",
        ],
        "coal_p_nom_mw": 39099.0,
    },
    "module14b_coal49_nuclear50": {
        "label": "MODULE14B-COAL49-NUCLEAR50",
        "paths": [
            RESULTS /
↪"elec_s_34_ec_lc1_NoCO2-1H-EAF-UC-OPC-LOW-GAS-OFFICIAL-FLEET-MODULE14B-COAL49-NUCLEAR50.
↪nc",
        ],
        "coal_p_nom_mw": 39099.0,
    },
}

```

```

SOLVE_LABELS = {key: cfg["label"] for key, cfg in SCENARIOS.items()}
NETWORK_PATHS = {key: cfg["paths"][0] for key, cfg in SCENARIOS.items()}

```

```

ESKOM_HOURLY_PATH = DATA_VALIDATION / "eskom_2023_hourly_clean.csv"
ESKOM_TARGETS_PATH = DATA_VALIDATION / "eskom_2023_targets_by_carrier.csv"
ESKOM_CAPACITY_PATH = DATA_AUDIT / "za_eskom_2023_capacity_anchors.csv"
CALIBRATED_DEMAND_PATH = ROOT / "data" / "ssp2-2.6" / "2030" /
↪"era5_2023_calibrated" / "Africa.csv"

```

```

WEEKLY_SCARCITY_PATH = DATA_VALIDATION / "za_2023_dispatch_pearson.csv"
MONTHLY_SCARCITY_PATH = DATA_VALIDATION / "za_2023_dispatch_pearson_monthly.csv"

CARRIER_COLORS = {
    "coal": "#4b4b4b",
    "nuclear": "#c84a4a",
    "ocgt": "#cf6a32",
    "sasol_coal": "#6f4e37",
    "sasol_gas": "#b56576",
    "hydro": "#4f93c7",
    "phs_generation": "#276fb7",
    "phs_pumping": "#6baed6",
    "wind": "#4c9f70",
    "solar_pv": "#e2b33c",
    "csp": "#f1992d",
    "biomass": "#8BC34A",
    "ror": "#4FC3F7",
    "load_shedding": "#b71c1c",
}

CARRIER_ORDER = list(CARRIER_COLORS)
SASOL_DIAGNOSTIC_CARRIERS = ["sasol_coal", "sasol_gas"]
ESKOM_COMPARABLE_GENERATION_CARRIERS = [
    c for c in CARRIER_ORDER
    if c not in {"phs_pumping", "load_shedding", "sasol_coal", "sasol_gas"}
]
PHYSICAL_GENERATION_CARRIERS = ESKOM_COMPARABLE_GENERATION_CARRIERS
MODEL_PHYSICAL_GENERATION_CARRIERS = [
    c for c in CARRIER_ORDER if c not in {"phs_pumping", "load_shedding"}
]
PLOTTED_GENERATION_CARRIERS = [c for c in CARRIER_ORDER if c not in
    ↪{"phs_pumping", "biomass", "ror"}]
DEEP_DIVE_CARRIERS = [
    "coal", "nuclear", "ocgt", "hydro", "phs_generation",
    "wind", "solar_pv", "csp", "load_shedding",
]

SERIES_STYLE = {
    "Eskom": {"color": "#111111", "linestyle": "-", "marker": "o", "hatch": ""},
    "LOW-GAS-OFFICIAL-FLEET": {"color": "#2ca02c", "linestyle": "-", "marker": "P", "hatch": "."},
    "MODULE14C-COAL485-NUCLEAR50-NO-VRE-NO-OCGT-CAP": {"color": "#7f7f7f", "linestyle": "-", "marker": "s", "hatch": "\\\""},
    "MODULE14C-COAL485-NUCLEAR50-VRE-ONLY": {"color": "#17becf", "linestyle": "-", "marker": "^", "hatch": "xx"},
    "MODULE14C-COAL485-NUCLEAR50": {"color": "#9467bd", "linestyle": "-", "marker": "D", "hatch": "/"},
}

```

```

        "MODULE14B-COAL49-NUCLEAR50": {"color": "#1f77b4", "linestyle": "-",
↪ "marker": "X", "hatch": "++"},
    }

MONTHS = np.arange(1, 13)
MONTH_LABELS = pd.date_range("2023-01-01", periods=12, freq="MS").strftime("%b")

def line_kwargs(label):
    return {k: v for k, v in SERIES_STYLE[label].items() if k != "hatch"}

if not ESKOM_HOURLY_PATH.exists():
    raise FileNotFoundError(f"Missing canonical Eskom hourly file:
↪ {ESKOM_HOURLY_PATH}")

eskom_raw = pd.read_csv(ESKOM_HOURLY_PATH, index_col=0, parse_dates=True).
↪ sort_index()
eskom_targets = pd.read_csv(ESKOM_TARGETS_PATH) if ESKOM_TARGETS_PATH.exists()
↪ else pd.DataFrame()
eskom_capacity_anchors = pd.read_csv(ESKOM_CAPACITY_PATH) if
↪ ESKOM_CAPACITY_PATH.exists() else pd.DataFrame()

def module13n_calibrated_demand_from_eskom(frame):
    unattributed = (
        frame["Dispatchable Generation"]
        - frame["Thermal Generation"]
        - frame["Nuclear Generation"]
        - frame["Eskom Gas Generation"]
        - frame["Eskom OCGT Generation"]
        - frame["Hydro Water Generation"]
        - frame["Pumped Water Generation"]
        - frame["Dispatchable IPP OCGT"]
    )
    return (
        frame["RSA Contracted Demand"]
        - frame["International Imports"]
        + frame["International Exports"]
        - frame["Other RE"]
        - unattributed
    )

calibrated_demand = module13n_calibrated_demand_from_eskom(eskom_raw)
if CALIBRATED_DEMAND_PATH.exists():
    calibrated_csv = pd.read_csv(CALIBRATED_DEMAND_PATH, sep=";")
    calibrated_csv["time"] = pd.to_datetime(calibrated_csv["time"])

```

```

        calibrated_csv = calibrated_csv.set_index("time")["Electricity demand"].
↪sort_index()
        max_abs_diff = (calibrated_demand.reindex(calibrated_csv.index) -
↪calibrated_csv).abs().max()
        if max_abs_diff > 1e-6:
            print(f"WARNING: formula demand differs from calibrated CSV by max_
↪{max_abs_diff:.6f} MW")
        calibrated_demand = calibrated_csv.reindex(eskom_raw.index)

path_status = pd.DataFrame(
    [{"artifact": "Eskom hourly clean", "path": ESKOM_HOURLY_PATH, "exists":
↪ESKOM_HOURLY_PATH.exists(), "role": "input"},
    {"artifact": "Module 13n calibrated demand", "path":
↪CALIBRATED_DEMAND_PATH, "exists": CALIBRATED_DEMAND_PATH.exists(), "role":
↪"input"}]
    + [
        {
            "artifact": cfg["label"],
            "path": candidate,
            "exists": candidate.exists(),
            "role": "candidate network",
        }
        for cfg in SCENARIOS.values()
        for candidate in cfg["paths"]
    ]
    + [
        {"artifact": "Eskom carrier targets", "path": ESKOM_TARGETS_PATH,
↪"exists": ESKOM_TARGETS_PATH.exists(), "role": "input"},
        {"artifact": "Eskom capacity anchors", "path": ESKOM_CAPACITY_PATH,
↪"exists": ESKOM_CAPACITY_PATH.exists(), "role": "input"},
    ]
)
path_status.style.format({"path": str})

```

[111]: <pandas.io.formats.style.Styler at 0x31744ba50>

```

[112]: def load_network(path: Path):
        if pypsa is None or not path.exists():
            return None
        return pypsa.Network(str(path))

def zero(index):
    return pd.Series(0.0, index=index)

def gen_dispatch(net, carriers):
    gens = net.generators[net.generators.carrier.isin(carriers)].index

```

```

if len(gens) == 0 or net.generators_t.p.empty:
    return zero(net.snapshots)
return net.generators_t.p[gens].sum(axis=1)

def storage_dispatch(net, carrier, mode="discharge"):
    """Positive storage_unit p is discharge; negative p is charging."""
    units = net.storage_units[net.storage_units.carrier == carrier].index
    if len(units) == 0 or net.storage_units_t.p.empty:
        return zero(net.snapshots)
    p = net.storage_units_t.p[units].sum(axis=1)
    if mode == "discharge":
        return p.clip(lower=0)
    if mode in ("pumping", "charge"):
        return (-p).clip(lower=0)
    return p

def csp_link_dispatch(net):
    """CSP electric output flows through carrier=='csp' links in this model."""
    if len(net.links) and (net.links.carrier == "csp").any():
        csp_links = net.links[net.links.carrier == "csp"].index
        if not net.links_t.p0.empty:
            return net.links_t.p0[csp_links].sum(axis=1).clip(lower=0)
    return gen_dispatch(net, ["csp"])

def model_timeseries(net):
    idx = pd.DatetimeIndex(net.snapshots)
    return pd.DataFrame({
        "coal": gen_dispatch(net, ["coal"]),
        "nuclear": gen_dispatch(net, ["nuclear"]),
        "ocgt": gen_dispatch(net, ["OCGT", "ocgt", "ocgt_diesel", "ocgt_gas"]),
        "sasol_coal": gen_dispatch(net, ["sasol_coal"]),
        "sasol_gas": gen_dispatch(net, ["sasol_gas"]),
        "hydro": storage_dispatch(net, "hydro", "discharge"),
        "phs_generation": storage_dispatch(net, "PHS", "discharge"),
        "phs_pumping": storage_dispatch(net, "PHS", "pumping"),
        "wind": gen_dispatch(net, ["onwind"]),
        "solar_pv": gen_dispatch(net, ["solar"]),
        "csp": csp_link_dispatch(net),
        "load_shedding": gen_dispatch(net, ["load shedding"]),
        "biomass": gen_dispatch(net, ["biomass"]),
        "ror": gen_dispatch(net, ["ror"]),
    }, index=idx).fillna(0.0)

def rsa_timeseries(net):
    """Extract dispatch timeseries from a PyPSA-RSA network.

```

```

RSA uses multi-period snapshots: MultiIndex (period, timestamp).
Carrier names differ from PyPSA-Earth conventions.
"""
if isinstance(net.snapshots, pd.MultiIndex):
    idx = pd.DatetimeIndex(net.snapshots.get_level_values(1))
else:
    idx = pd.DatetimeIndex(net.snapshots)

def rsa_gen(carriers):
    gens = net.generators[net.generators.carrier.isin(carriers)].index
    if len(gens) == 0 or net.generators_t.p.empty:
        return pd.Series(0.0, index=idx)
    return pd.Series(net.generators_t.p.reindex(columns=gens, fill_value=0.
↪0).sum(axis=1).values, index=idx)

return pd.DataFrame({
    "coal": rsa_gen(["coal"]),
    "nuclear": rsa_gen(["nuclear"]),
    "ocgt": rsa_gen(["ocgt_diesel"]),
    "sasol_coal": rsa_gen(["sasol_coal"]),
    "sasol_gas": rsa_gen(["sasol_gas"]),
    "hydro": rsa_gen(["hydro", "hydro_import"]),
    "phs_generation": pd.Series(0.0, index=idx),
    "phs_pumping": pd.Series(0.0, index=idx),
    "wind": rsa_gen(["wind"]),
    "solar_pv": rsa_gen(["solar_pv"]),
    "csp": rsa_gen(["solar_csp"]),
    "load_shedding": rsa_gen(["load_shedding"]),
    "biomass": rsa_gen(["bioenergy"]),
    "ror": pd.Series(0.0, index=idx),
}, index=idx).fillna(0.0)

def _get_column(frame, column, index):
    if column in frame:
        return frame[column].reindex(index).fillna(0.0)
    return pd.Series(0.0, index=index)

def eskom_timeseries(index):
    e = eskom_raw.reindex(index)
    out = pd.DataFrame(index=index)
    out["coal"] = _get_column(e, "Thermal Generation", index)
    out["nuclear"] = _get_column(e, "Nuclear Generation", index)
    out["ocgt"] = _get_column(e, "Eskom OCGT Generation", index) + ↵
↪_get_column(e, "Dispatchable IPP OCGT", index)
    out["hydro"] = _get_column(e, "Hydro Water Generation", index)
    out["phs_generation"] = _get_column(e, "Pumped Water Generation", index)
    out["phs_pumping"] = _get_column(e, "Pumped Water SCO Pumping", index).abs()

```

```

out["wind"] = _get_column(e, "Wind", index)
out["solar_pv"] = _get_column(e, "PV", index)
out["csp"] = _get_column(e, "CSP", index)
out["load_shedding"] = (
    _get_column(e, "Manual Load_Reduction(MLR)", index)
    + _get_column(e, "ILS Usage", index)
    + _get_column(e, "IOS Excl ILS and MLR", index)
)
return out.reindex(columns=CARRIER_ORDER, fill_value=0.0)

def annual_gwh(series):
    return float(series.sum() / 1e3)

def safe_corr(func, left, right):
    frame = pd.concat([left, right], axis=1).dropna()
    if len(frame) < 2 or frame.iloc[:, 0].std() == 0 or frame.iloc[:, 1].std()
    == 0:
        return np.nan
    return float(func(frame.iloc[:, 0], frame.iloc[:, 1])[0])

def pearson(left, right):
    return safe_corr(pearsonr, left, right)

def spearman(left, right):
    return safe_corr(spearmanr, left, right)

def mae(left, right):
    frame = pd.concat([left, right], axis=1).dropna()
    if frame.empty:
        return np.nan
    return float((frame.iloc[:, 0] - frame.iloc[:, 1]).abs().mean())

def rmse(left, right):
    frame = pd.concat([left, right], axis=1).dropna()
    if frame.empty:
        return np.nan
    return float(np.sqrt(((frame.iloc[:, 0] - frame.iloc[:, 1]) ** 2).mean()))

def pct_delta(model_gwh, eskom_gwh):
    return ((model_gwh - eskom_gwh) / eskom_gwh * 100.0) if abs(eskom_gwh) >
    1e-9 else np.nan

def style_energy_table(df, delta_cols):
    gwh_cols = [c for c in df.columns if c.endswith("_GWh") or c == "eskom_GWh"]
    return (
        df.style
        .format({col: "{:,.0f}" for col in gwh_cols}, na_rep="")
    )

```

```

        .format({col: "{:+.1f}%" for col in delta_cols}, na_rep="")
        .background_gradient(cmap="RdYlGn_r", subset=delta_cols)
    )

def style_metric_table(df):
    formats = {
        "Annual GWh": "{:,.0f}",
        "Delta vs Eskom GWh": "{:+.0f}",
        "Delta %": "{:+.1f}%",
        "Pearson r (hourly)": "{:.3f}",
        "Spearman r (hourly)": "{:.3f}",
        "Pearson r (weekly)": "{:.3f}",
        "RMSE MW": "{:,.0f}",
        "MAE MW": "{:,.0f}",
    }
    return df.style.format({k: v for k, v in formats.items() if k in df.
↪columns}, na_rep="")

def monthly_gwh(frame):
    return frame.resample("ME").sum() / 1e3

def weekly_gwh(series):
    return series.resample("W").sum() / 1e3

```

```

[113]: def coal_p_nom_mw(net):
        if net is None or "carrier" not in net.generators:
            return np.nan
        return float(net.generators.loc[net.generators.carrier.eq("coal"), "p_nom"].
↪sum())

def candidate_matches_signature(net, cfg):
    target = cfg.get("coal_p_nom_mw")
    if target is None:
        return True, ""
    actual = coal_p_nom_mw(net)
    if np.isfinite(actual) and abs(actual - float(target)) <=
↪COAL_P_NOM_TOLERANCE_MW:
        return True, f"coal_p_nom={actual:.1f} MW"
    return False, f"coal_p_nom={actual:.1f} MW does not match expected {target:.
↪1f} MW"

def load_registered_network(solve, cfg):
    notes = []
    for candidate in cfg["paths"]:
        if pypsa is None:
            return None, candidate, "PyPSA unavailable"
        if not candidate.exists():

```

```

        notes.append(f"missing: {candidate.name}")
        continue
    net = load_network(candidate)
    ok, reason = candidate_matches_signature(net, cfg)
    if ok:
        return net, candidate, reason or "loaded"
    notes.append(f"skipped {candidate.name}: {reason}")
    return None, cfg["paths"][0], "; ".join(notes)

networks = {}
resolved_network_paths = {}
load_notes = {}
for solve, cfg in SCENARIOS.items():
    net, selected_path, note = load_registered_network(solve, cfg)
    resolved_network_paths[solve] = selected_path
    load_notes[solve] = note
    if net is None:
        print(f"WARNING: skipping {cfg['label']}: {note}")
        continue
    networks[solve] = net

series = {}
for solve, net in networks.items():
    if solve == "rsa_2023bm":
        model = rsa_timeseries(net)
    else:
        model = model_timeseries(net)
    reference = eskom_timeseries(model.index)
    series[solve] = {"model": model, "eskom": reference}

loaded_status = pd.DataFrame([
    {
        "solve": SOLVE_LABELS[solve],
        "path": resolved_network_paths[solve],
        "loaded": solve in networks,
        "snapshots": len(networks[solve].snapshots) if solve in networks else 0,
        "coal_p_nom_mw": coal_p_nom_mw(networks[solve]) if solve in networks
    }
])
loaded_status.style.format({"path": str, "snapshots": "{:,}", "coal_p_nom_mw":
    "{:,.1f}"})

```

[113]: <pandas.io.formats.style.Styler at 0x326df9c50>

1.2 1. System overview table

Summarise the annual carrier-level energy balance before moving into detailed time-series diagnostics. The total row reports physical generation only; load shedding is kept as a separate scarcity comparison.

1.2.1 1.1 Annual energy balance table

```
[114]: def build_annual_balance():
    rows = []
    reference = next(iter(series.values()))["eskom"] if series else
    eskom_timeseries(eskom_raw.index)
    for carrier in CARRIER_ORDER:
        row = {"carrier": carrier, "Eskom_GWh": annual_gwh(reference[carrier])}
        for solve, label in SOLVE_LABELS.items():
            if solve in series:
                model_gwh = annual_gwh(series[solve]["model"][carrier])
                row[f"{label}_GWh"] = model_gwh
                row[f"{label}_Delta_pct"] = pct_delta(model_gwh,
    row["Eskom_GWh"])
            else:
                row[f"{label}_GWh"] = np.nan
                row[f"{label}_Delta_pct"] = np.nan
        rows.append(row)

    total = {"carrier": "TOTAL_PHYSICAL_GENERATION"}
    value_cols = ["Eskom_GWh"] + [f"{label}_GWh" for label in SOLVE_LABELS.
    values()]
    generation_rows = [r for r in rows if r["carrier"] in
    PHYSICAL_GENERATION_CARRIERS]
    for col in value_cols:
        total[col] = np.nansum([r[col] for r in generation_rows])
    for label in SOLVE_LABELS.values():
        total[f"{label}_Delta_pct"] = pct_delta(total[f"{label}_GWh"],
    total["Eskom_GWh"])
    rows.append(total)
    return pd.DataFrame(rows).set_index("carrier")

annual_balance = build_annual_balance()
delta_cols = [f"{label}_Delta_pct" for label in SOLVE_LABELS.values()]
style_energy_table(annual_balance, delta_cols)
```

```
[114]: <pandas.io.formats.style.Styler at 0x3232d8090>
```

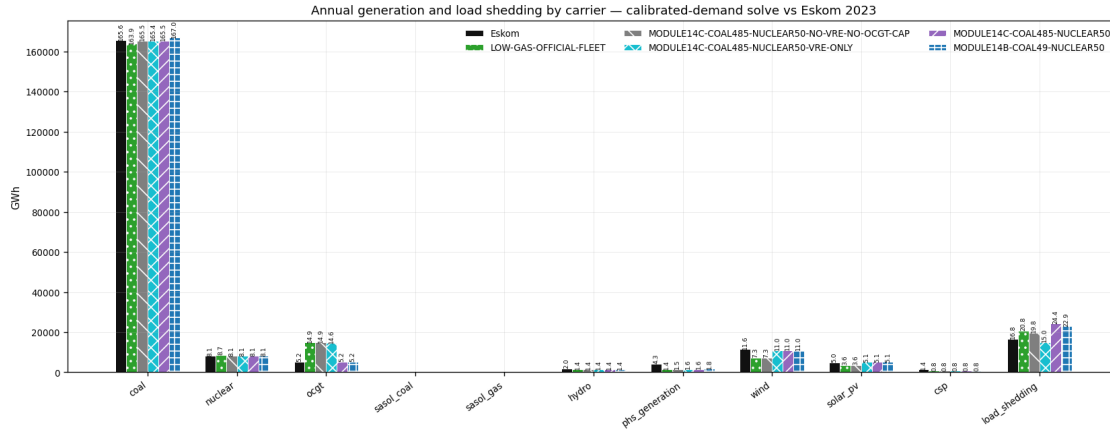
1.2.2 1.2 Annual generation by carrier

```
[115]: chart_carriers = PLOTTED_GENERATION_CARRIERS
plot_data = annual_balance.loc[chart_carriers, ["Eskom_GWh"] + [f"{label}_GWh"]
    ↪ for label in SOLVE_LABELS.values()]]
plot_data.columns = ["Eskom"] + list(SOLVE_LABELS.values())

fig, ax = plt.subplots(figsize=(14, 5.5))
x = np.arange(len(plot_data.index))
n_cols = len(plot_data.columns)
width = min(0.12, 0.82 / max(n_cols, 1))
for i, column in enumerate(plot_data.columns):
    offset = (i - (n_cols - 1) / 2) * width
    bars = ax.bar(
        x + offset,
        plot_data[column].values,
        width,
        label=column,
        color=SERIES_STYLE[column]["color"],
        hatch=SERIES_STYLE[column]["hatch"],
        edgecolor="white",
        linewidth=0.6,
    )
    for bar, value in zip(bars, plot_data[column].values):
        if np.isfinite(value) and value > 0:
            ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height(),
                ↪ f"{value / 1000:.1f}",
                    ha="center", va="bottom", rotation=90, fontsize=5.5)

ax.set_xticks(x)
ax.set_xticklabels(plot_data.index, rotation=35, ha="right")
ax.set_ylabel("GWh")
ax.set_title("Annual generation and load shedding by carrier -
    ↪ calibrated-demand solve vs Eskom 2023")
ax.legend(loc="upper right", ncol=3, frameon=False)
plt.tight_layout()

FIGURES_ACCEPTANCE = ROOT / "doc" / "za_validation" / "figures" /
    ↪ "12_acceptance"
FIGURES_ACCEPTANCE.mkdir(parents=True, exist_ok=True)
fig.savefig(FIGURES_ACCEPTANCE / "annual_generation_bars_calibrated_demand.
    ↪ png", dpi=140, bbox_inches="tight")
plt.show()
print("Saved → annual_generation_bars_calibrated_demand.png")
```



Saved → annual_generation_bars_calibrated_demand.png

1.2.3 1.3 Total dispatch fit progression — July 2023

Multi-panel view of total physical dispatch (model vs Eskom) for each solve across July 2023. Each panel shows RMSE and bias in the title; the improvement in fit as calibration layers are added is visible in the shrinking error bands.

```
[116]: if series:
    july = slice("2023-07-01", "2023-07-31")

    def total_disp(ts):
        return ts[PHYSICAL_GENERATION_CARRIERS].sum(axis=1)

    eskom_ref_total = total_disp(next(iter(series.values()))["eskom"])

    panels = [
        (label, total_disp(series[solve]["model"]), eskom_ref_total)
        for solve, label in SOLVE_LABELS.items()
        if solve in series
    ]

    fig, axes = plt.subplots(len(panels), 1, figsize=(14, 2.1 * len(panels)),
                             sharex=True)
    if len(panels) == 1:
        axes = [axes]

    for ax, (label, model, ref) in zip(axes, panels):
        mod_jul = model.loc[july]
        ref_jul = ref.loc[july]
        over = mod_jul > ref_jul
```

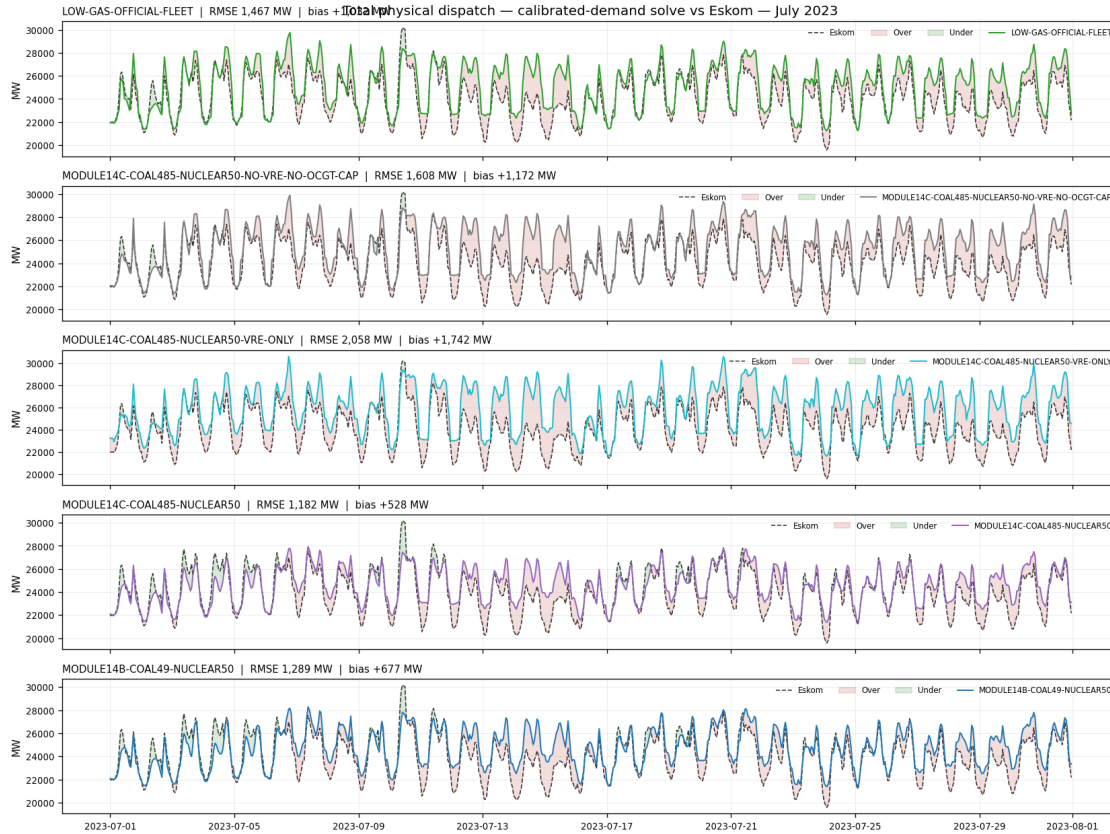
```

        ax.plot(ref_jul.index, ref_jul.values, color="black", lw=1.0, ls="--",
↪alpha=0.75, label="Eskom")
        ax.fill_between(ref_jul.index, mod_jul.values, ref_jul.values,
                        where=over.values, interpolate=True, color="#c14b4b",
↪alpha=0.18, label="Over")
        ax.fill_between(ref_jul.index, mod_jul.values, ref_jul.values,
                        where=(~over).values, interpolate=True,
↪color="#3a8a3a", alpha=0.18, label="Under")
        ax.plot(mod_jul.index, mod_jul.values,
↪color=SERIES_STYLE[label]["color"], lw=1.2, label=label)
        rmse_val = rmse(mod_jul, ref_jul)
        bias_val = float((mod_jul - ref_jul).mean())
        ax.set_ylabel("MW")
        ax.set_title(f"{label} | RMSE {rmse_val:,.0f} MW | bias {bias_val:
↪+, .0f} MW", loc="left", fontsize=9)
        ax.legend(loc="upper right", ncol=4, frameon=False, fontsize=7)
        ax.ticklabel_format(axis="y", style="plain", useOffset=False)
        ax.grid(axis="y", alpha=0.25)

    axes[0].set_title("Total physical dispatch - calibrated-demand solve vs
↪Eskom - July 2023", loc="center", fontsize=12)
    plt.tight_layout()

    FIGURES_DISPATCH = ROOT / "doc" / "za_validation" / "figures" /
↪"12_dispatch_calibration"
    FIGURES_DISPATCH.mkdir(parents=True, exist_ok=True)
    fig.savefig(FIGURES_DISPATCH / "total_dispatch_calibrated_demand_panel.
↪png", dpi=140, bbox_inches="tight")
    plt.show()
    print(f"Saved → total_dispatch_calibrated_demand_panel.png ({len(pannels)}
↪pannels)")

```



Saved → total_dispatch_calibrated_demand_panel.png (5 panels)

1.3 2. Carrier-by-carrier deep dive

Compare each technology against Eskom across hourly July dispatch, monthly totals, diurnal shape, and fit metrics. These views separate timing error from annual energy-level error.

```
[117]: def carrier_metrics(carrier):
    reference = next(iter(series.values()))["eskom"][carrier] if series else
    eskom_timeseries(eskom_raw.index)[carrier]
    eskom_gwh = annual_gwh(reference)
    rows = []
    for solve, label in SOLVE_LABELS.items():
        if solve not in series:
            continue
        model = series[solve]["model"][carrier]
        model_gwh = annual_gwh(model)
        rows.append({
            "Solve": label,
            "Annual GWh": model_gwh,
            "Delta vs Eskom GWh": model_gwh - eskom_gwh,
```

```

        "Delta %": pct_delta(model_gwh, eskom_gwh),
        "Pearson r (hourly)": pearson(model, reference),
        "Spearman r (hourly)": spearman(model, reference),
        "Pearson r (weekly)": pearson(weekly_gwh(model),
weekly_gwh(reference)),
        "RMSE MW": rmse(model, reference),
        "MAE MW": mae(model, reference),
    })
    return pd.DataFrame(rows)

def plot_hourly_july(carrier):
    if not series:
        return

    reference = next(iter(series.values()))["eskom"][carrier].loc["2023-07-01":
weekly_gwh(reference)]
    panels = []
    for solve, label in SOLVE_LABELS.items():
        if solve in series:
            model = series[solve]["model"][carrier].loc[reference.index]
            panels.append((label, model, reference))

    combined = pd.concat([reference] + [model for _, model, _ in panels])
    y_max = float(combined.max()) if len(combined) else 0.0
    y_top = y_max * 1.08 if y_max > 0 else 1.0

    fig, axes = plt.subplots(len(panels), 1, figsize=(13, 2.05 * len(panels)),
sharex=True)
    if len(panels) == 1:
        axes = [axes]
    for ax, (label, model, ref) in zip(axes, panels):
        over = model > ref
        ax.plot(ref.index, ref.values, color="black", lw=1.15, ls="--", alpha=0.
8, label="Eskom")
        ax.fill_between(ref.index, model.values, ref.values, where=over.values,
interpolate=True, color="#c14b4b", alpha=0.20, label="Model > Eskom")
        ax.fill_between(ref.index, model.values, ref.values, where=(~over).
values, interpolate=True, color="#3a8a3a", alpha=0.20, label="Model < Eskom")
        ax.plot(model.index, model.values, color=CARRIER_COLORS[carrier], lw=1.
35, label=label)
        ax.set_ylim(0, y_top)
        ax.ticklabel_format(axis="y", style="plain", useOffset=False)
        ax.set_ylabel("MW")
        ax.set_title(f"{label} | MAE {mae(model, ref):.0f} MW | bias {(model -
ref).mean():+.0f} MW", loc="left", fontsize=9)
        ax.grid(axis="y", alpha=0.25)

```

```

        ax.legend(loc="upper right", ncol=4, frameon=False, fontsize=7)
        axes[0].set_title(f"{carrier} - Hourly dispatch July 2023", loc="center",
↪ fontsize=12)
        plt.tight_layout()
        plt.show()

def plot_monthly(carrier):
    reference = next(iter(series.values()))["eskom"] if series else
↪ eskom_timeseries(eskom_raw.index)
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(MONTHS, monthly_gwh(reference)[carrier].values, label="Eskom",
↪ **line_kwargs("Eskom"))
    for solve, label in SOLVE_LABELS.items():
        if solve in series:
            ax.plot(MONTHS, monthly_gwh(series[solve]["model"])[carrier].
↪ values, label=label, **line_kwargs(label))
    ax.set_xticks(MONTHS)
    ax.set_xticklabels(MONTH_LABELS)
    ax.set_ylabel("GWh")
    ax.set_title(f"{carrier} - Monthly energy totals")
    ax.legend(loc="upper right", frameon=False)
    plt.tight_layout()
    plt.show()

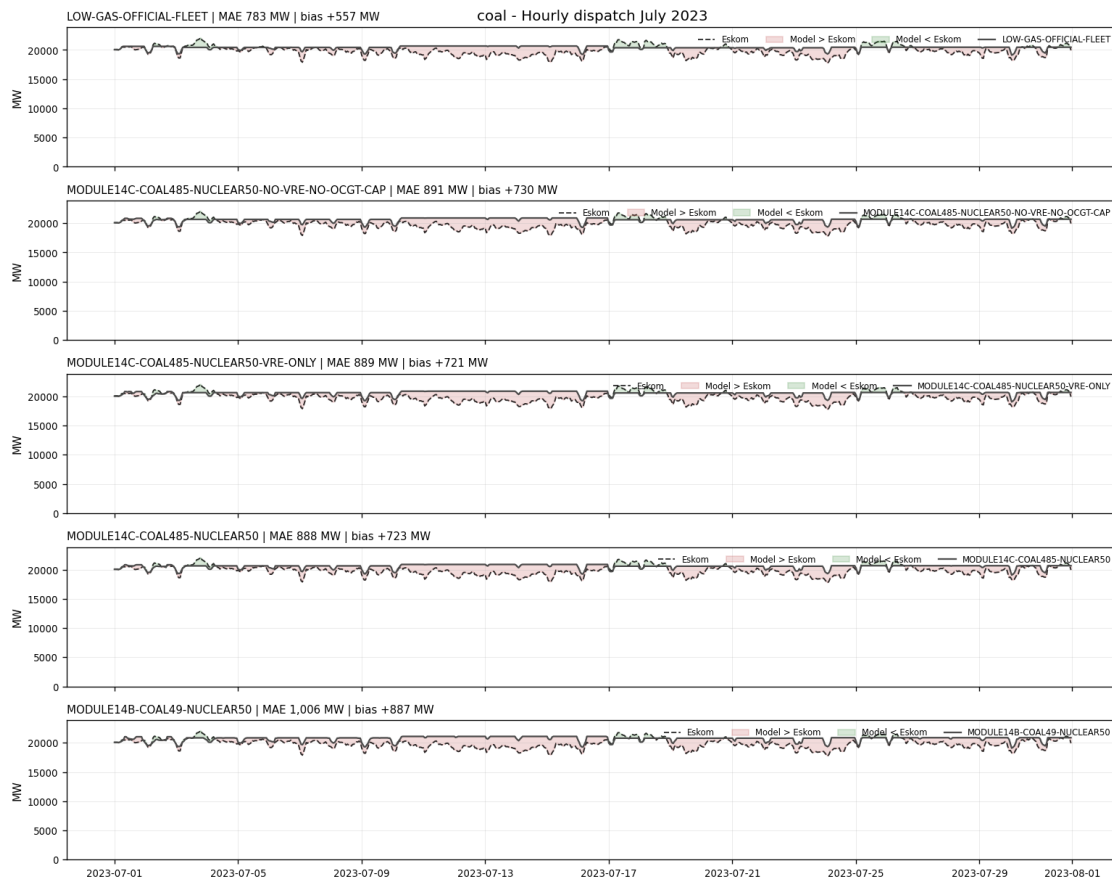
def plot_diurnal(carrier):
    reference = next(iter(series.values()))["eskom"] if series else
↪ eskom_timeseries(eskom_raw.index)
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(reference.groupby(reference.index.hour)[carrier].mean(),
↪ label="Eskom", **line_kwargs("Eskom"))
    for solve, label in SOLVE_LABELS.items():
        if solve in series:
            profile = series[solve]["model"].groupby(series[solve]["model"].
↪ index.hour)[carrier].mean()
            ax.plot(profile.index, profile.values, label=label,
↪ **line_kwargs(label))
    ax.set_xticks(range(0, 24, 2))
    ax.set_xlabel("Hour of day")
    ax.set_ylabel("Average MW")
    ax.set_title(f"{carrier} - Diurnal profile annual average")
    ax.legend(loc="upper right", frameon=False)
    plt.tight_layout()
    plt.show()

```

1.3.1 2.1. coal

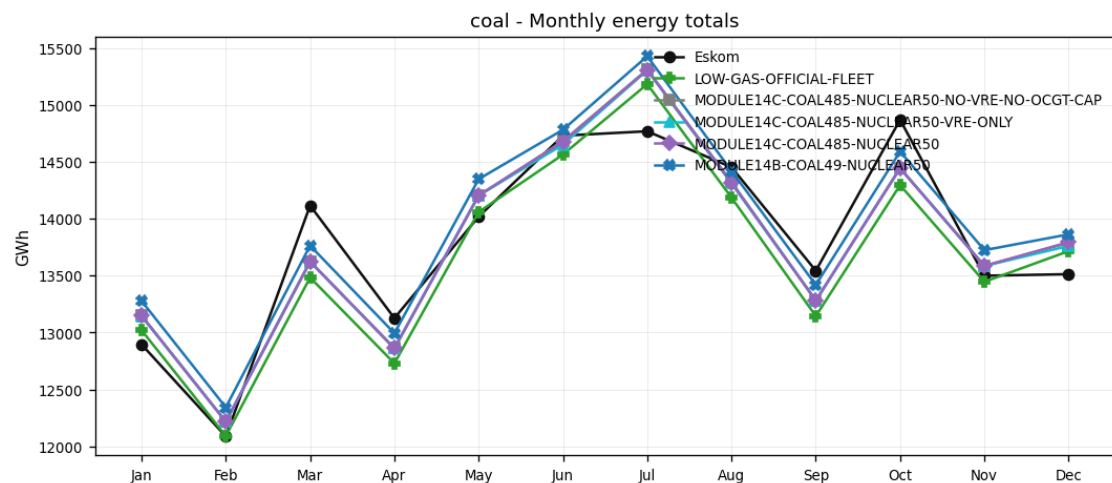
2.1.1 Hourly dispatch - July 2023

```
[118]: plot_hourly_july("coal")
```



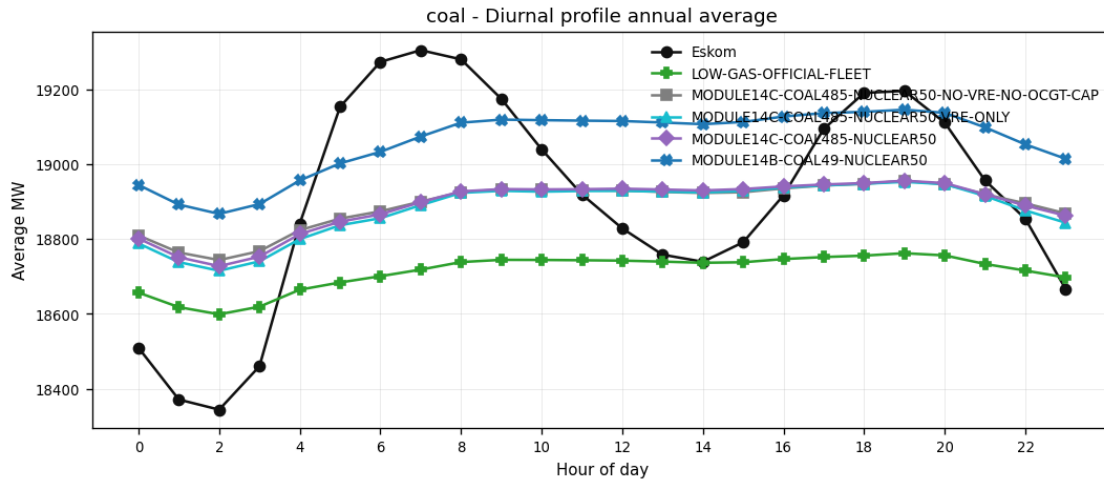
2.1.2 Monthly energy totals

```
[119]: plot_monthly("coal")
```



2.1.3 Diurnal profile annual average

```
[120]: plot_diurnal("coal")
```



2.1.4 Carrier metrics table Coal metrics are reported for the single Module 13n calibrated-demand LOW-GAS-OFFICIAL-FLEET solve against Eskom CY2023 historical coal dispatch.

```
[121]: coal_metric_table = carrier_metrics("coal")
style_metric_table(coal_metric_table)
```

```
[121]: <pandas.io.formats.style.Styler at 0x323277750>
```

1.3.2 2.2. nuclear

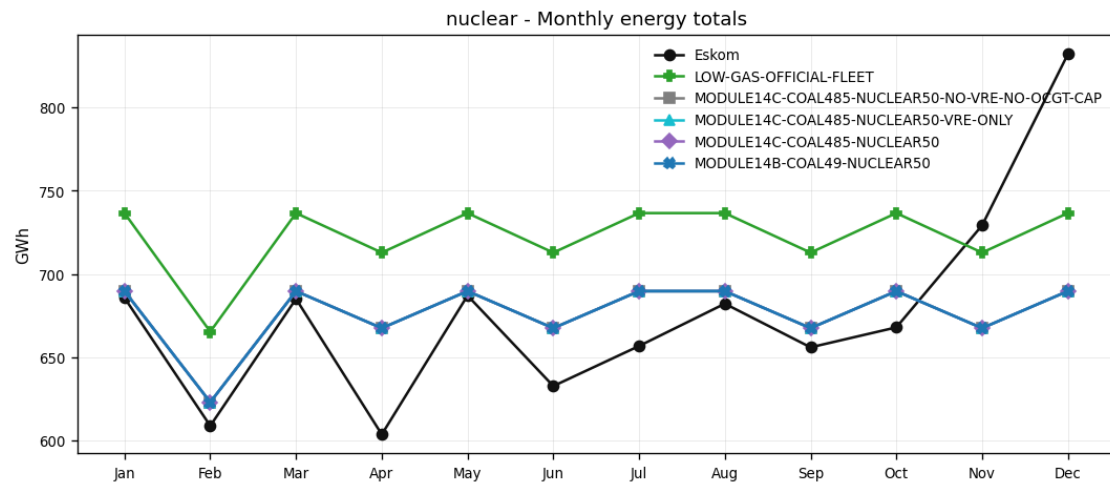
2.2.1 Hourly dispatch - July 2023

```
[122]: plot_hourly_july("nuclear")
```



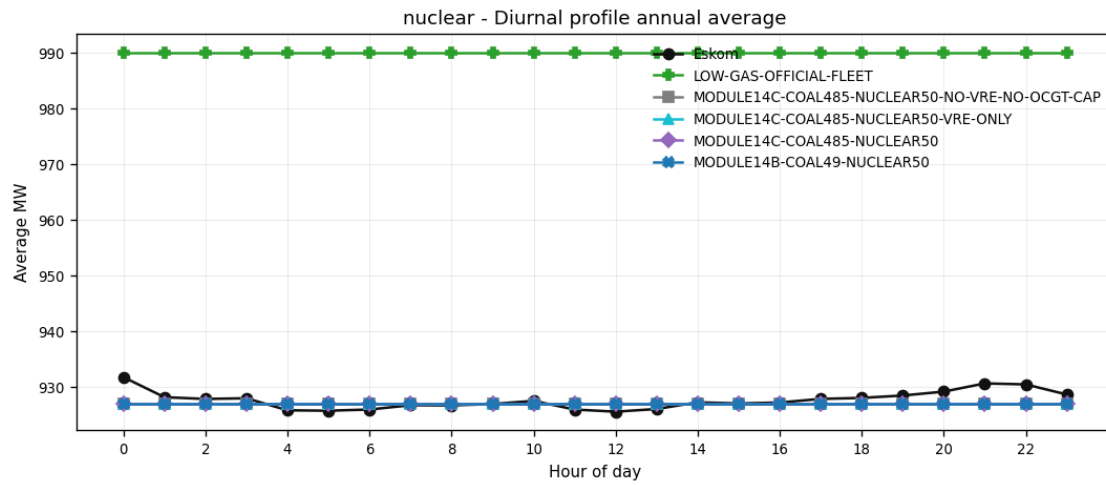
2.2.2 Monthly energy totals

[123]: `plot_monthly("nuclear")`



2.2.3 Diurnal profile annual average

```
[124]: plot_diurnal("nuclear")
```



2.2.4 Carrier metrics table

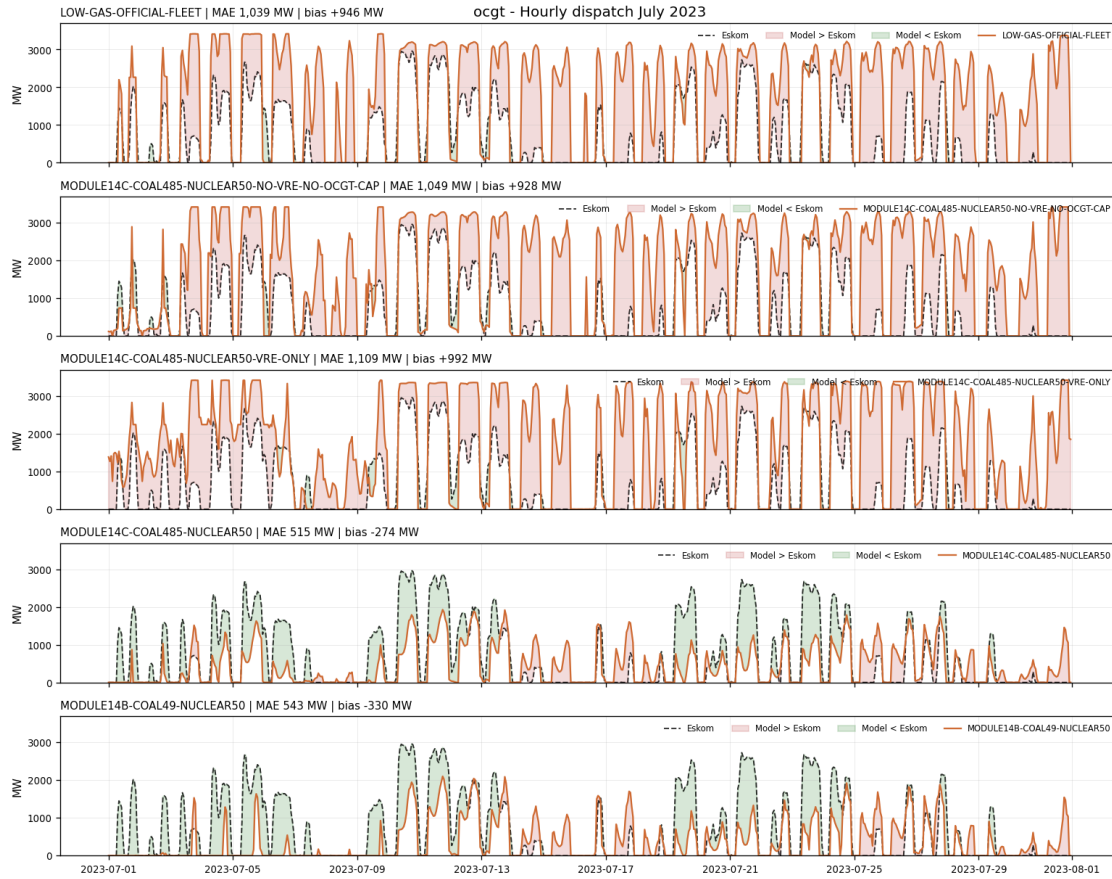
```
[125]: style_metric_table(carrier_metrics("nuclear"))
```

```
[125]: <pandas.io.formats.style.Styler at 0x32303a2d0>
```

1.3.3 2.3. ocgt

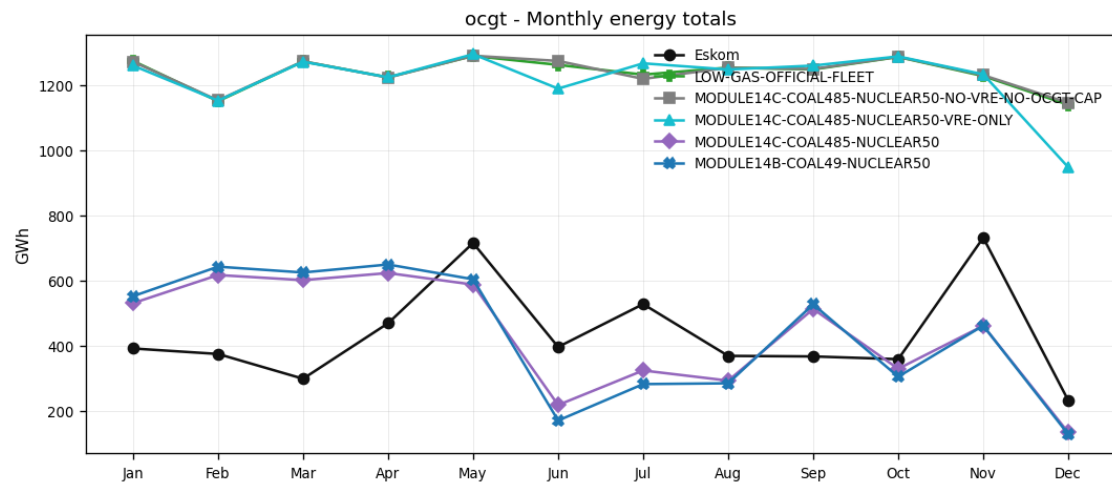
2.3.1 Hourly dispatch - July 2023

```
[126]: plot_hourly_july("ocgt")
```



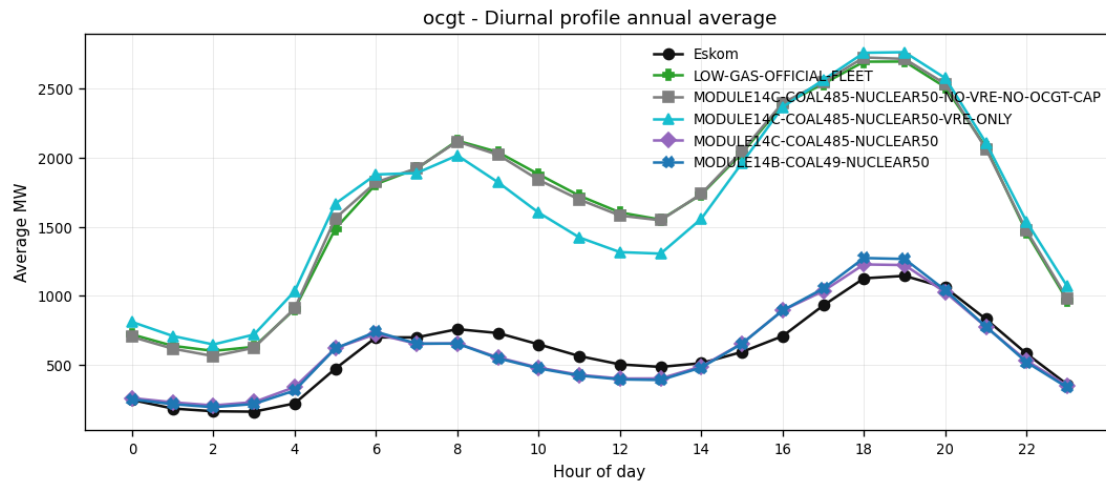
2.3.2 Monthly energy totals

[127]: `plot_monthly("ocgt")`



2.3.3 Diurnal profile annual average

```
[128]: plot_diurnal("ocgt")
```



2.3.4 Carrier metrics table

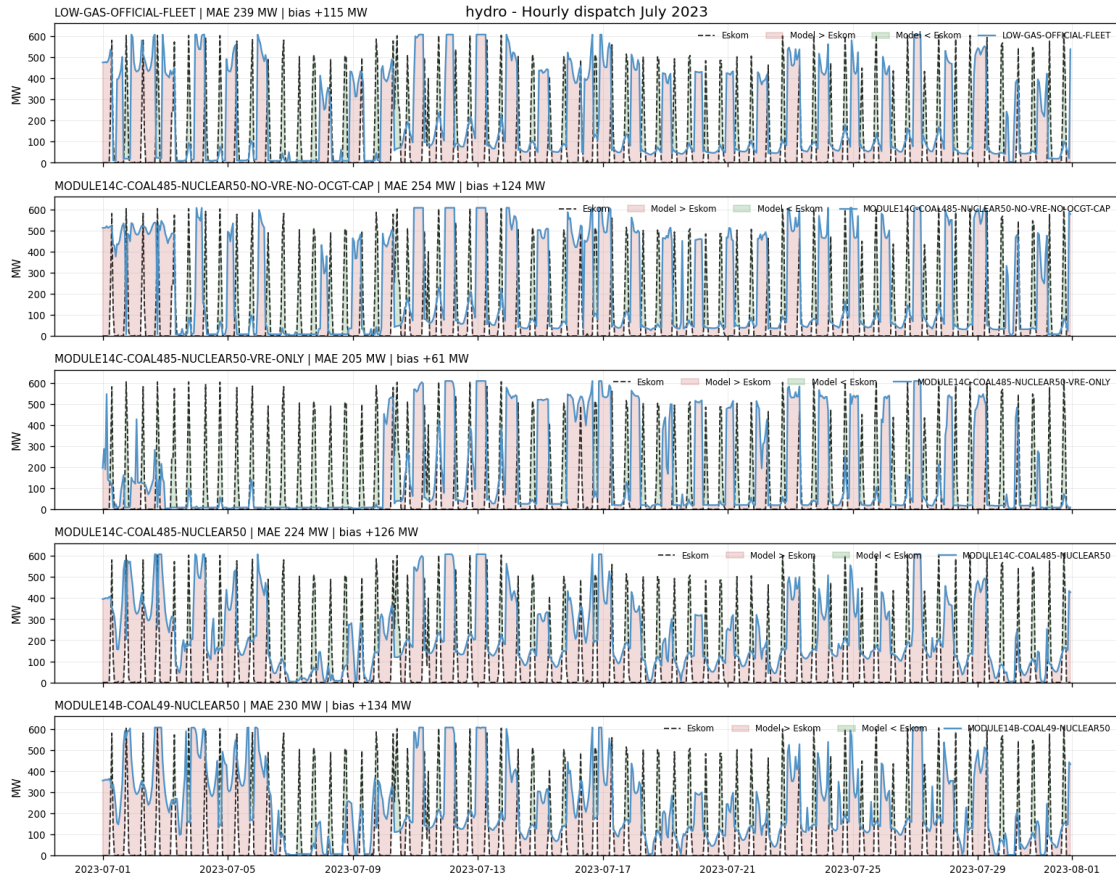
```
[129]: style_metric_table(carrier_metrics("ocgt"))
```

```
[129]: <pandas.io.formats.style.Styler at 0x32309a2d0>
```

1.3.4 2.4. hydro

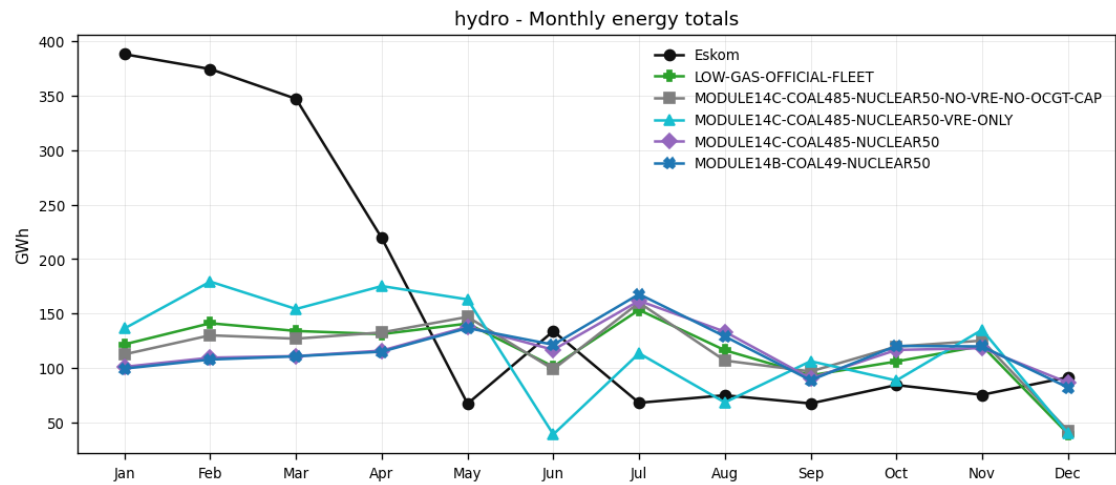
2.4.1 Hourly dispatch - July 2023

```
[130]: plot_hourly_july("hydro")
```



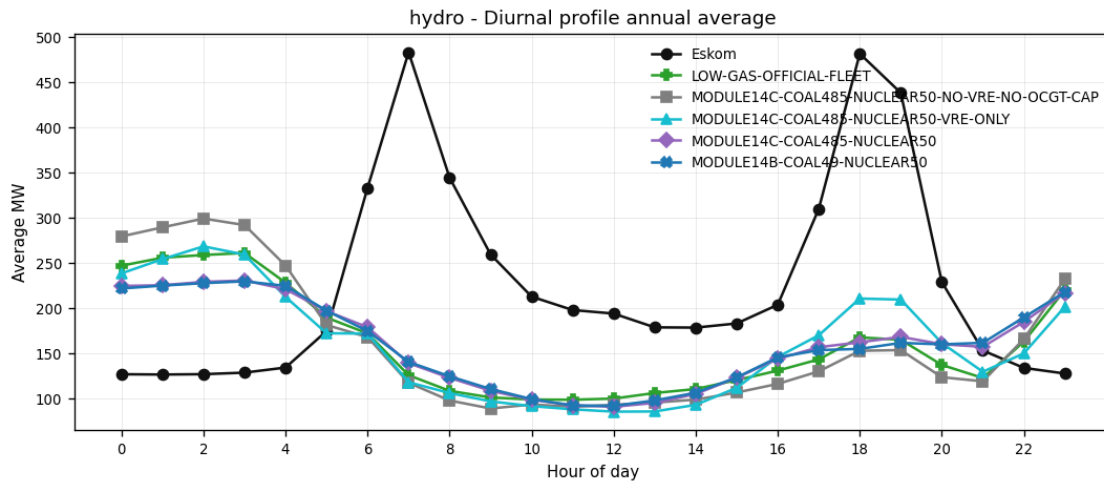
2.4.2 Monthly energy totals

```
[131]: plot_monthly("hydro")
```



2.4.3 Diurnal profile annual average

```
[132]: plot_diurnal("hydro")
```



2.4.4 Carrier metrics table

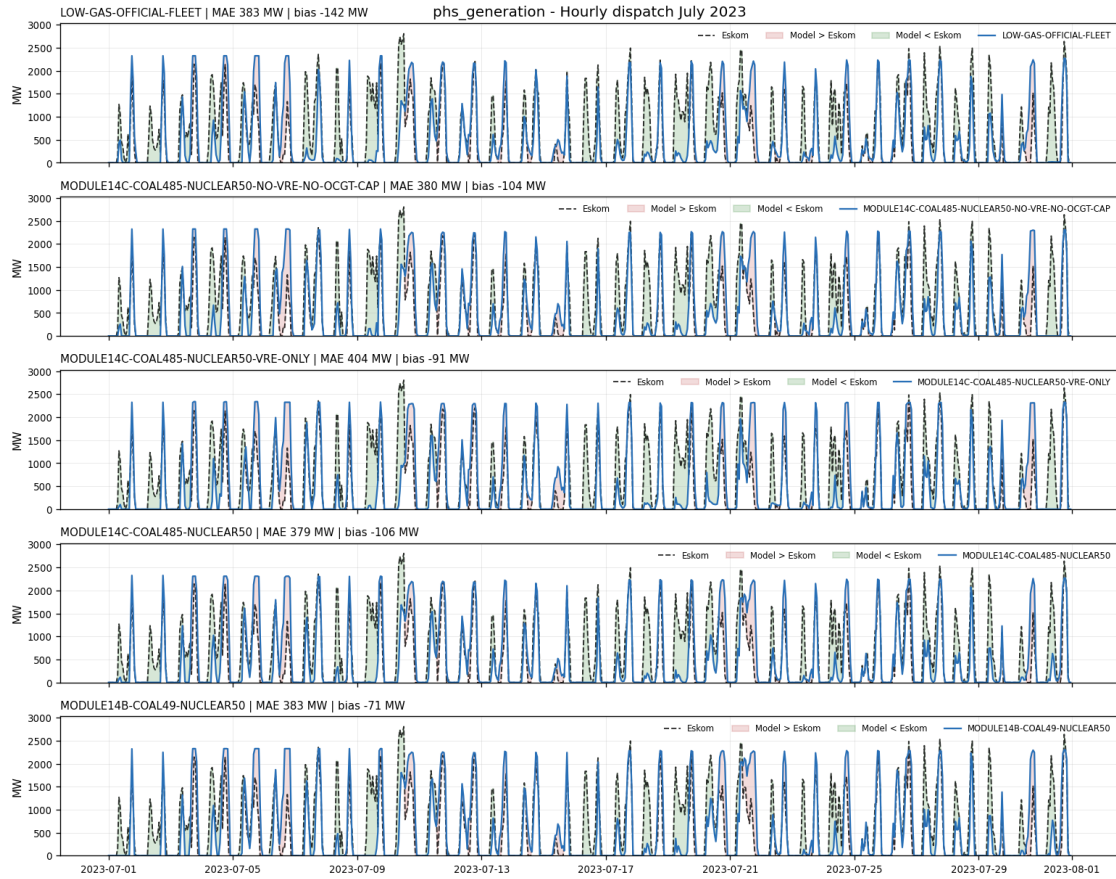
```
[133]: style_metric_table(carrier_metrics("hydro"))
```

```
[133]: <pandas.io.formats.style.Styler at 0x30d97d0d0>
```

1.3.5 2.5. phs_generation

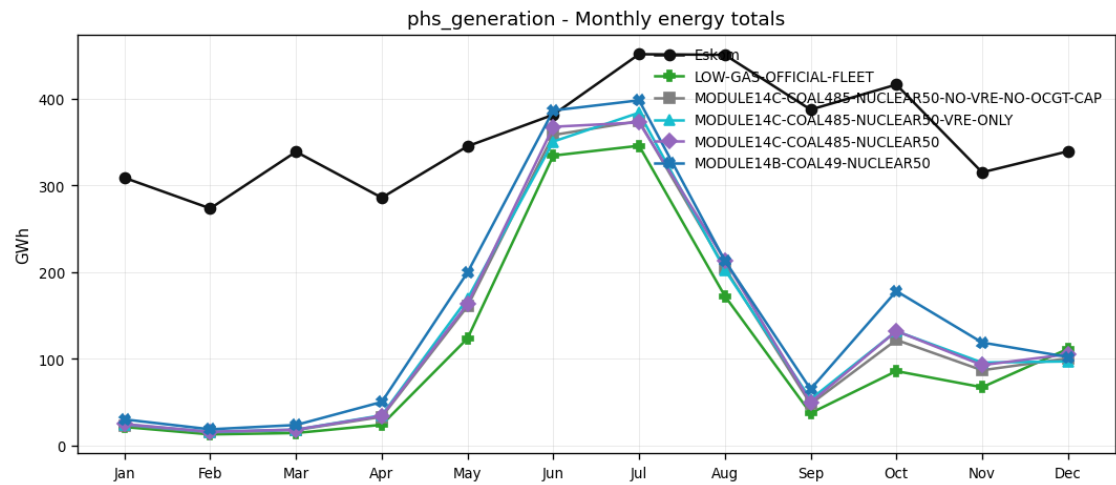
2.5.1 Hourly dispatch - July 2023

```
[134]: plot_hourly_july("phs_generation")
```



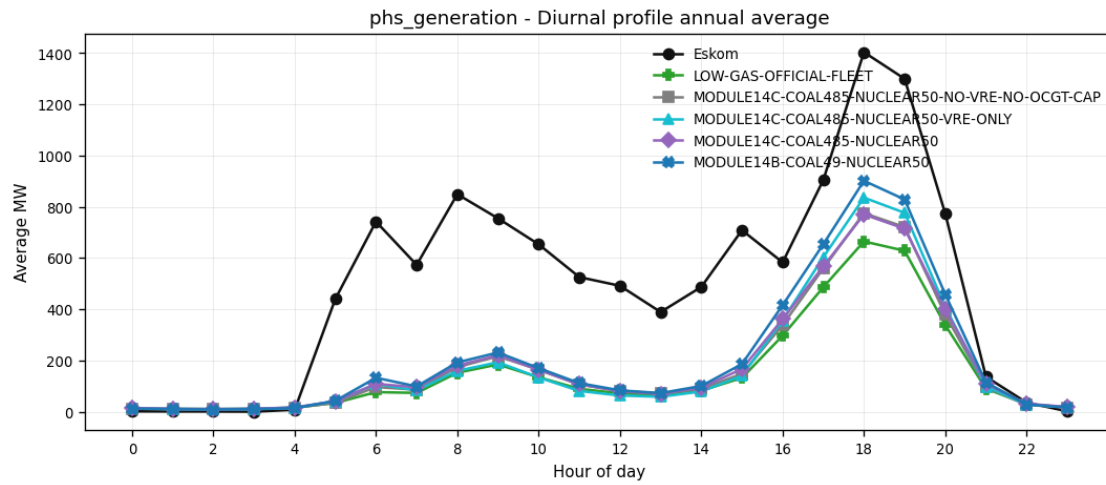
2.5.2 Monthly energy totals

```
[135]: plot_monthly("phs_generation")
```



2.5.3 Diurnal profile annual average

```
[136]: plot_diurnal("phs_generation")
```



2.5.4 Carrier metrics table

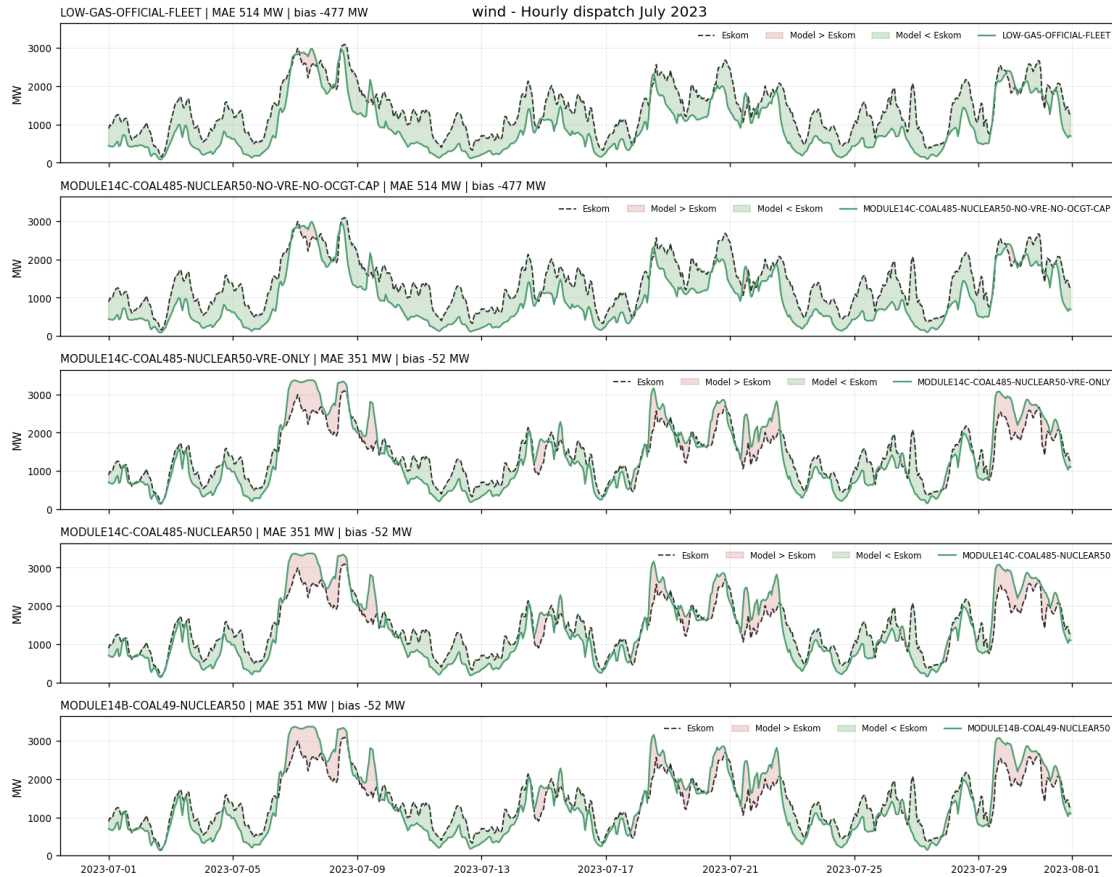
```
[137]: style_metric_table(carrier_metrics("phs_generation"))
```

```
[137]: <pandas.io.formats.style.Styler at 0x322da22d0>
```

1.3.6 2.6. wind

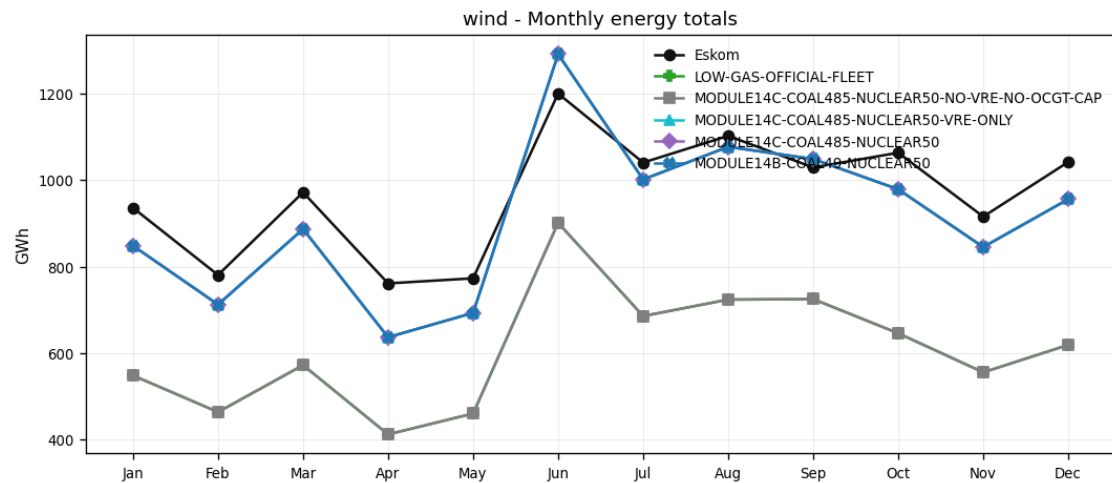
2.6.1 Hourly dispatch - July 2023

```
[138]: plot_hourly_july("wind")
```



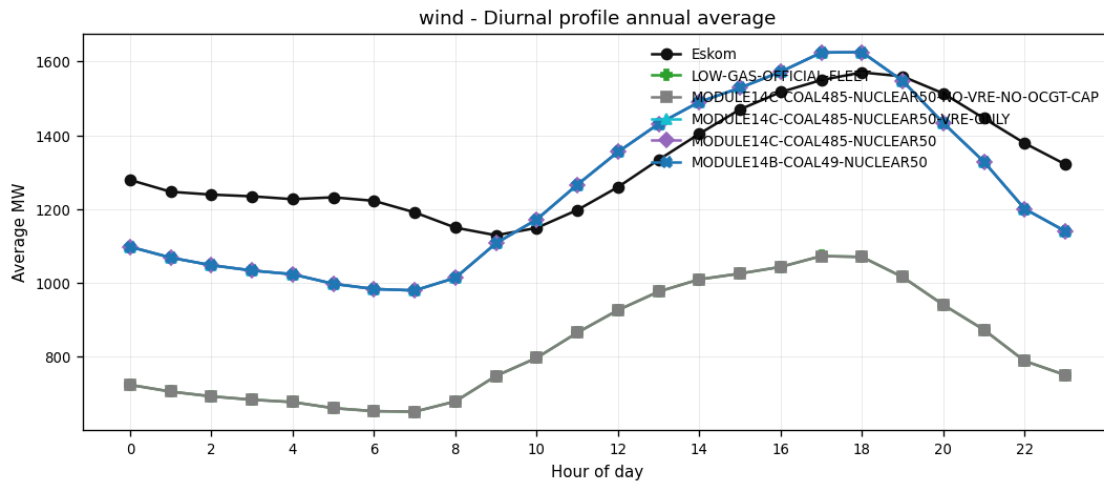
2.6.2 Monthly energy totals

[139]: `plot_monthly("wind")`



2.6.3 Diurnal profile annual average

```
[140]: plot_diurnal("wind")
```



2.6.4 Carrier metrics table

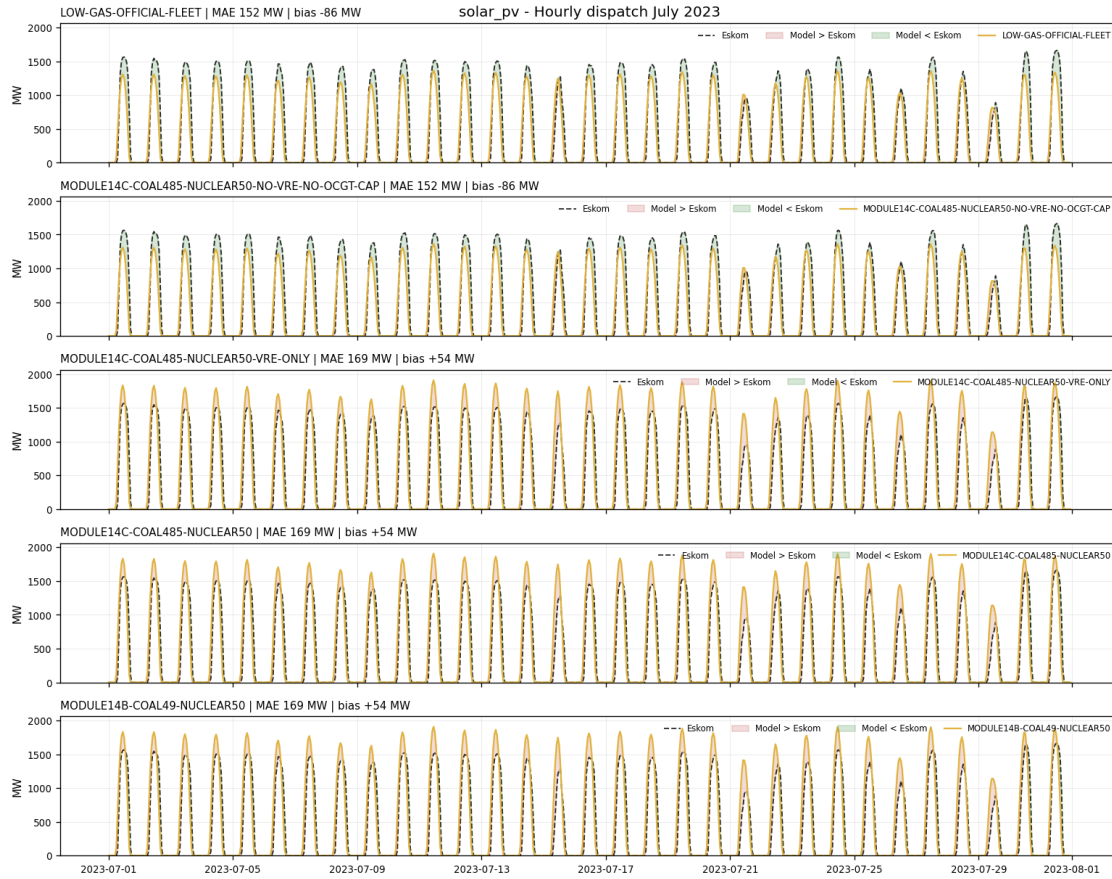
```
[141]: style_metric_table(carrier_metrics("wind"))
```

```
[141]: <pandas.io.formats.style.Styler at 0x3217d50d0>
```

1.3.7 2.7. solar_pv

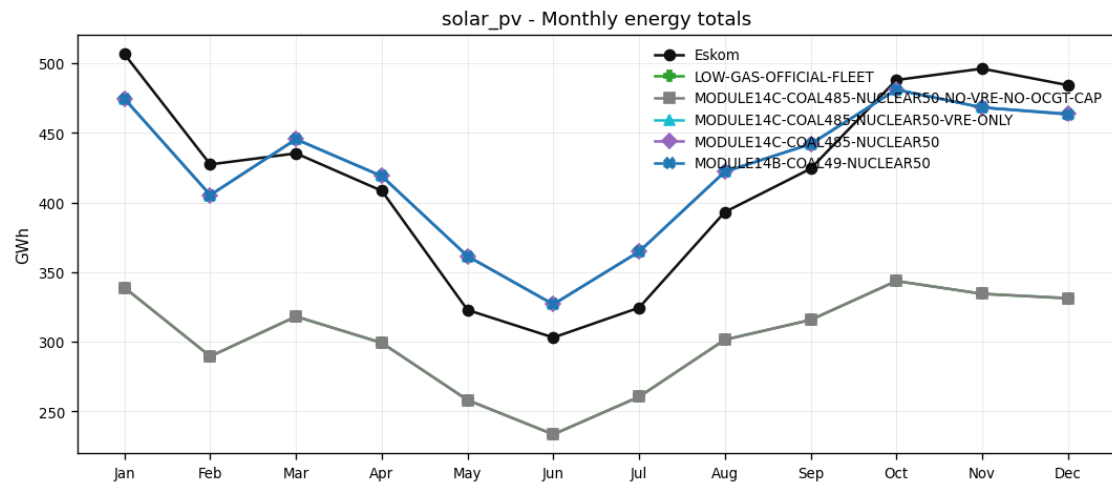
2.7.1 Hourly dispatch - July 2023

```
[142]: plot_hourly_july("solar_pv")
```



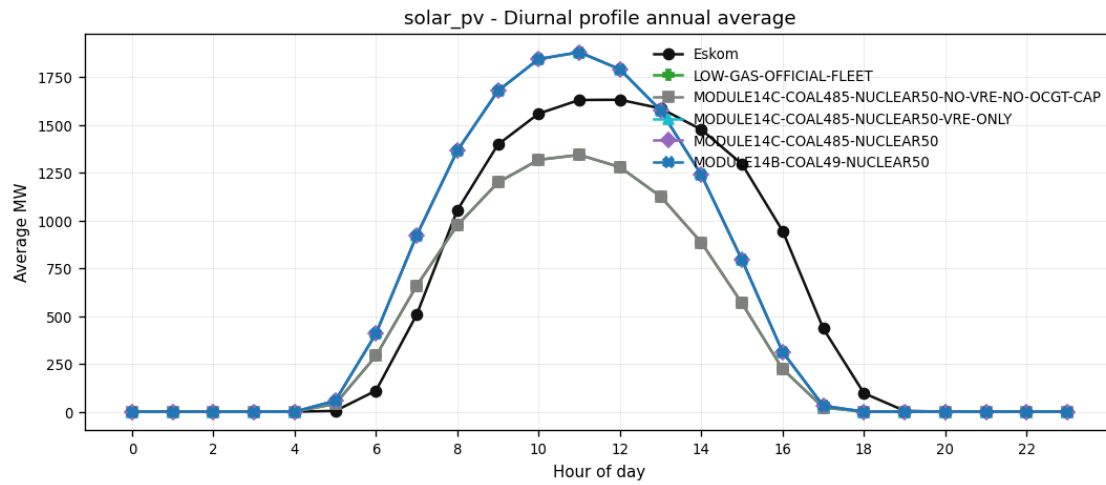
2.7.2 Monthly energy totals

[143]: `plot_monthly("solar_pv")`



2.7.3 Diurnal profile annual average

```
[144]: plot_diurnal("solar_pv")
```



2.7.4 Carrier metrics table

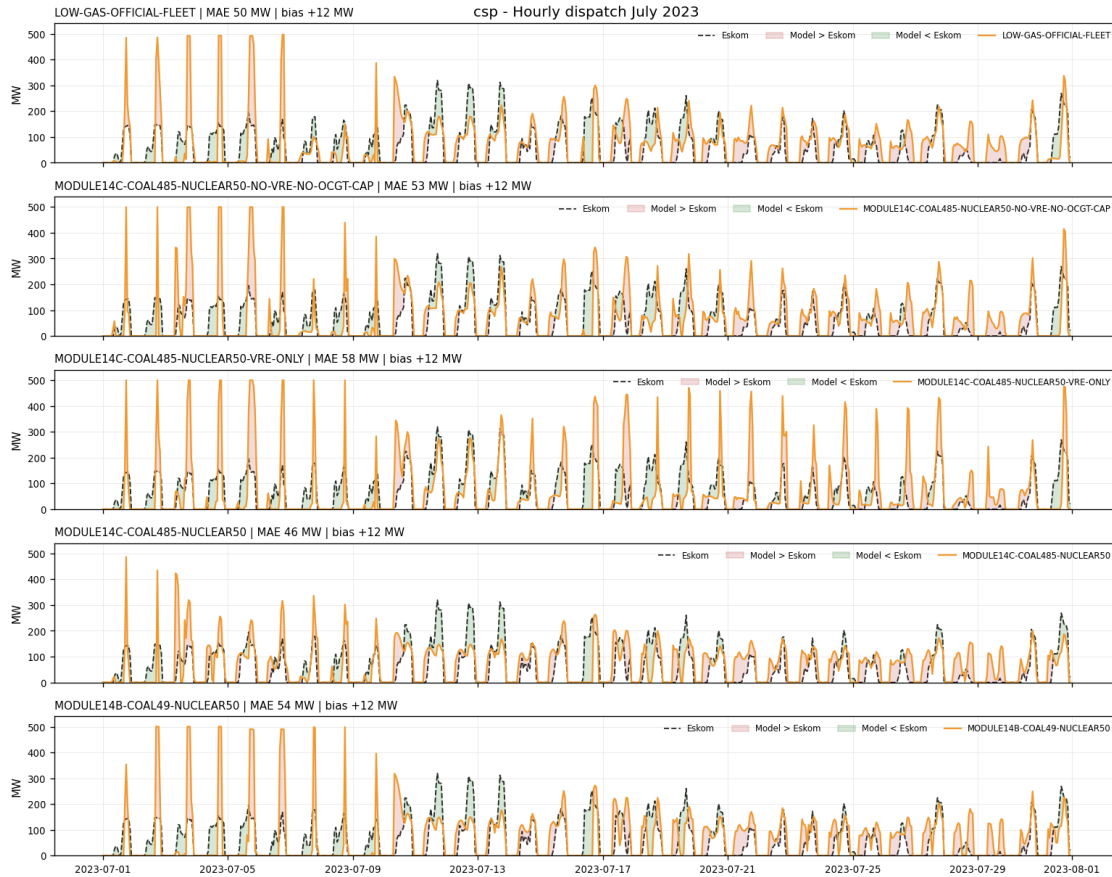
```
[145]: style_metric_table(carrier_metrics("solar_pv"))
```

```
[145]: <pandas.io.formats.style.Styler at 0x322ed0550>
```

1.3.8 2.8. csp

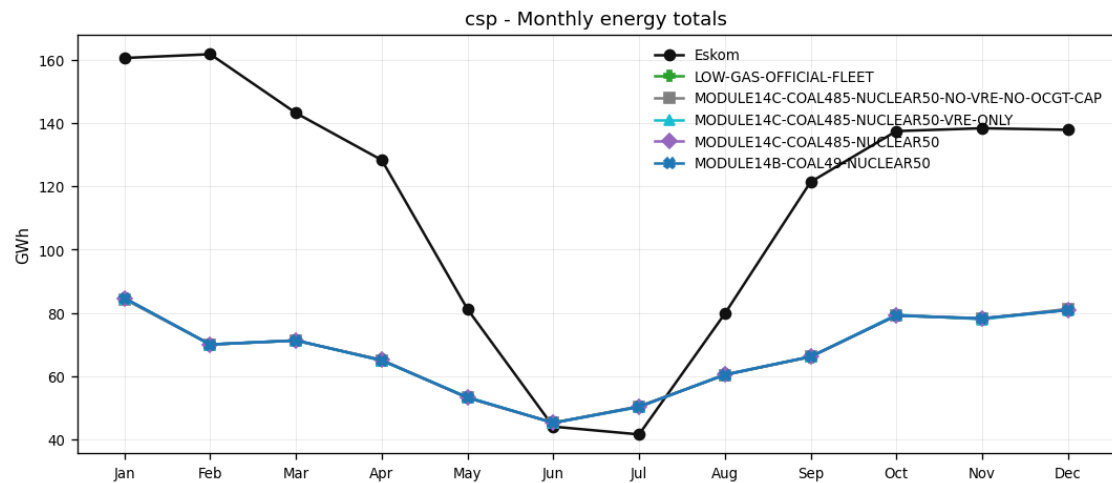
2.8.1 Hourly dispatch - July 2023

```
[146]: plot_hourly_july("csp")
```



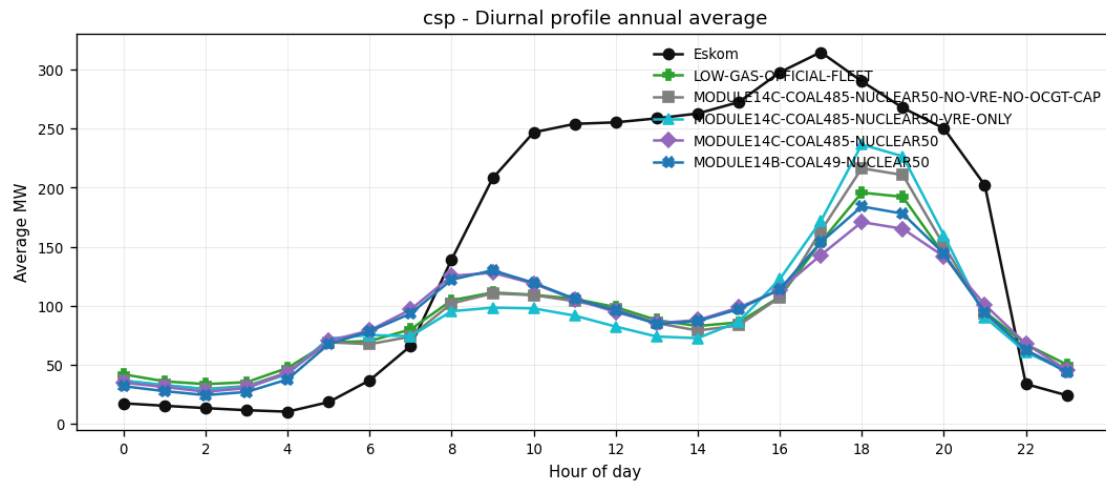
2.8.2 Monthly energy totals

[147]: `plot_monthly("csp")`



2.8.3 Diurnal profile annual average

```
[148]: plot_diurnal("csp")
```



2.8.4 Carrier metrics table

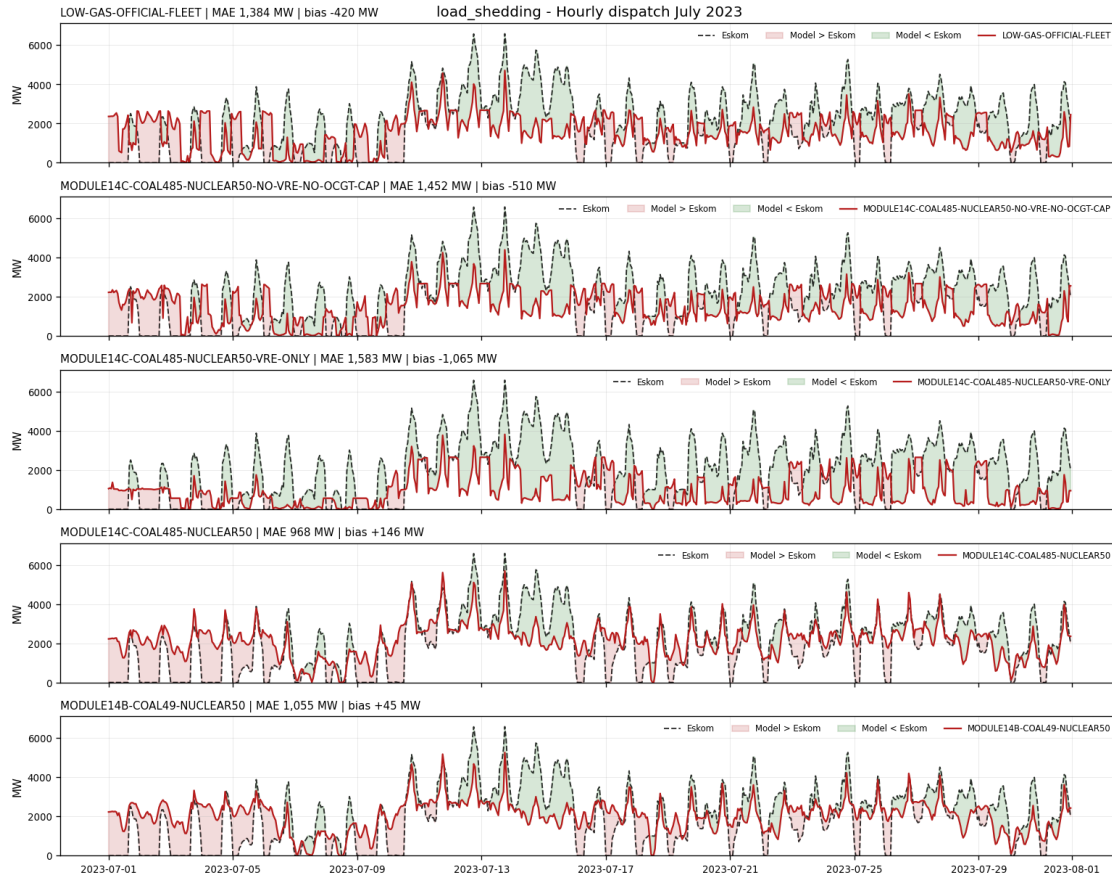
```
[149]: style_metric_table(carrier_metrics("csp"))
```

```
[149]: <pandas.io.formats.style.Styler at 0x32316a2d0>
```

1.3.9 2.9. load_shedding

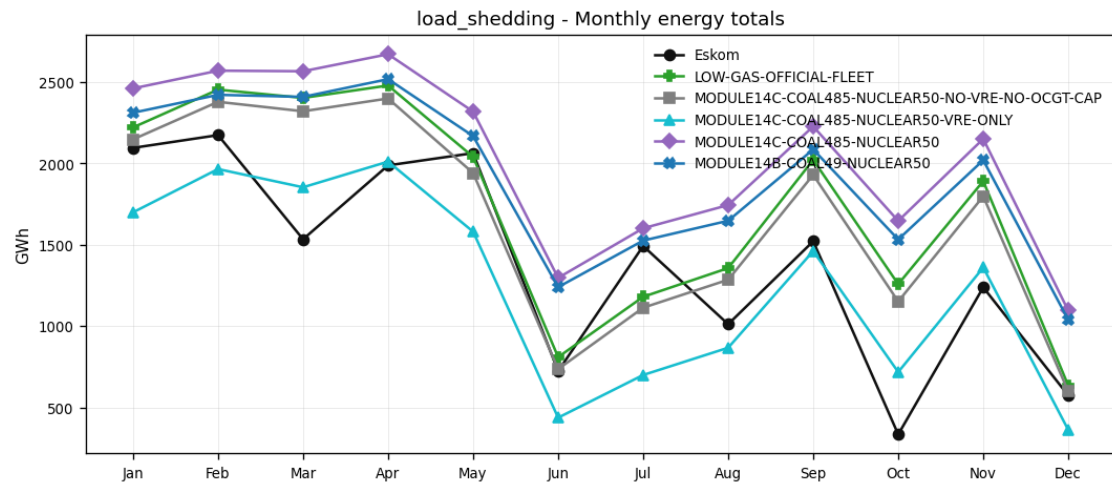
2.9.1 Hourly dispatch - July 2023

```
[150]: plot_hourly_july("load_shedding")
```



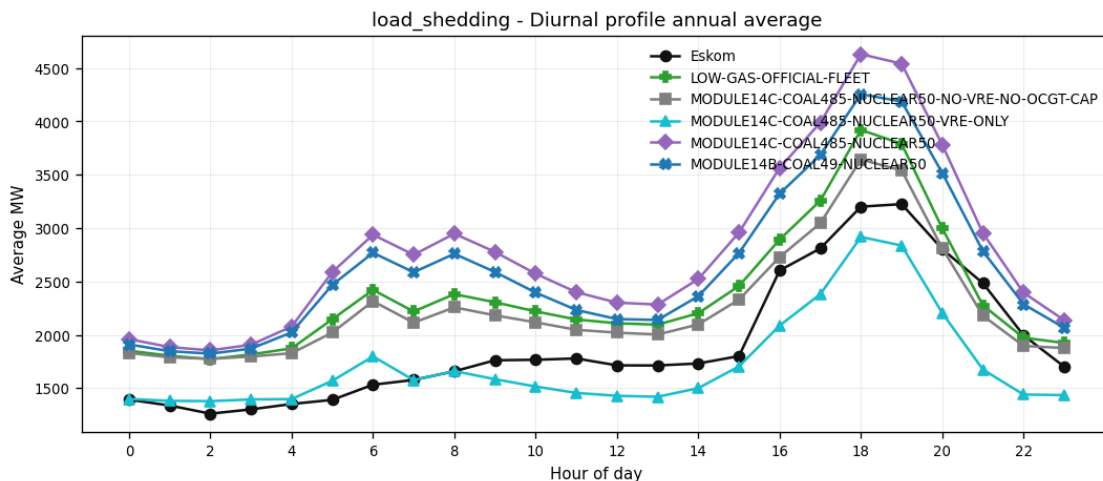
2.9.2 Monthly energy totals

```
[151]: plot_monthly("load_shedding")
```



2.9.3 Diurnal profile annual average

```
[152]: plot_diurnal("load_shedding")
```



2.9.4 Carrier metrics table

```
[153]: style_metric_table(carrier_metrics("load_shedding"))
```

```
[153]: <pandas.io.formats.style.Styler at 0x323240550>
```

Module 13n / 14c ablation note: this copy loads the accepted calibrated-demand LOW-GAS-OFFICIAL-FLEET solve, the Module 14c coal48.5+nuclear50 ablation sequence, and the MODULE14B-COAL49-NUCLEAR50 comparator. RSA-BM and other historical comparison cases are omitted so every plot remains a direct Eskom-vs-current-calibration comparison.

1.4 3. Scarcity proxy analysis

Evaluate whether the model reproduces the timing of scarcity using OCGT plus load shedding as a combined proxy. Weekly and monthly aggregation reduce hourly noise and focus on operational stress periods.

1.4.1 3.1 Weekly scarcity correlation table

```
[154]: def scarcity_series(frame):
    return frame["ocgt"] + frame["load_shedding"]

def load_existing_scarcity_metrics():
    if not (WEEKLY_SCARCITY_PATH.exists() and MONTHLY_SCARCITY_PATH.exists()):
        return None
    weekly = pd.read_csv(WEEKLY_SCARCITY_PATH)
    monthly = pd.read_csv(MONTHLY_SCARCITY_PATH)
    if "solve" not in weekly or "solve" not in monthly:
```

```

        return None
    rows = []
    for solve, label in SOLVE_LABELS.items():
        if solve not in series:
            continue
        w = weekly[weekly["solve"].eq(solve)]
        m = monthly[monthly["solve"].eq(solve)]
        if w.empty or m.empty:
            return None
        model_week = weekly_gwh(scarcity_series(series[solve]["model"]))
        eskom_week = weekly_gwh(scarcity_series(series[solve]["eskom"]))
        rows.append({
            "Solve": label,
            "Pearson r (weekly)": float(w.iloc[0].get("r_combined_scarcity", np.
↪nan)),
            "Spearman r (weekly)": float(w.iloc[0].get("rho_combined_scarcity",
↪np.nan)),
            "Pearson r (monthly)": float(m.iloc[0].get("r_combined_scarcity",
↪np.nan)),
            "Spearman r (monthly)": float(m.iloc[0].
↪get("rho_combined_scarcity", np.nan)),
            "MAE (GWh/week)": mae(model_week, eskom_week),
        })
    return pd.DataFrame(rows)

def compute_scarcity_metrics():
    rows = []
    for solve, label in SOLVE_LABELS.items():
        if solve not in series:
            continue
        model = scarcity_series(series[solve]["model"])
        reference = scarcity_series(series[solve]["eskom"])
        model_week = weekly_gwh(model)
        eskom_week = weekly_gwh(reference)
        model_month = model.resample("ME").sum() / 1e3
        eskom_month = reference.resample("ME").sum() / 1e3
        rows.append({
            "Solve": label,
            "Pearson r (weekly)": pearson(model_week, eskom_week),
            "Spearman r (weekly)": spearman(model_week, eskom_week),
            "Pearson r (monthly)": pearson(model_month, eskom_month),
            "Spearman r (monthly)": spearman(model_month, eskom_month),
            "MAE (GWh/week)": mae(model_week, eskom_week),
        })
    return pd.DataFrame(rows)

```

```

# Compute directly from the loaded networks so the current EAF-UC solve is
↳reflected
# even when older cached validation CSVs are still present.
scarcity_metrics = compute_scarcity_metrics()

scarcity_metrics.style.format({
    "Pearson r (weekly)": "{:.3f}",
    "Spearman r (weekly)": "{:.3f}",
    "Pearson r (monthly)": "{:.3f}",
    "Spearman r (monthly)": "{:.3f}",
    "MAE (GWh/week)": "{:,.0f}",
}, na_rep="").background_gradient(cmap="RdYlGn", subset=["Pearson r (weekly)",
↳"Spearman r (weekly)", "Pearson r (monthly)", "Spearman r (monthly)"])

```

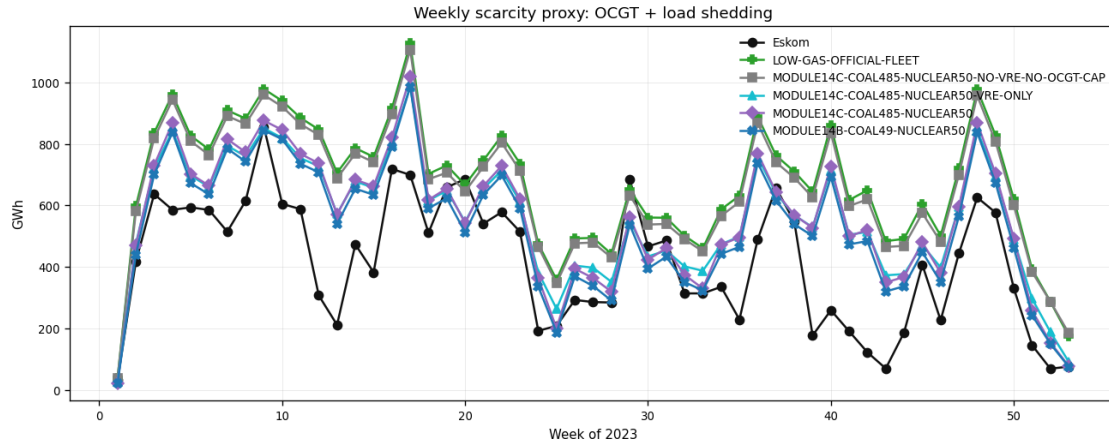
[154]: <pandas.io.formats.style.Styler at 0x322e979d0>

1.4.2 3.2 Weekly scarcity time series

```

[155]: fig, ax = plt.subplots(figsize=(11, 4.5))
reference = scarcity_series(next(iter(series.values()))["eskom"] if series else
↳eskom_timeseries(eskom_raw.index))
eskom_week = weekly_gwh(reference)
week_numbers = np.arange(1, len(eskom_week) + 1)
ax.plot(week_numbers, eskom_week.values, label="Eskom", **line_kwargs("Eskom"))
for solve, label in SOLVE_LABELS.items():
    if solve in series:
        model_week = weekly_gwh(scarcity_series(series[solve]["model"]))
        ax.plot(week_numbers[:len(model_week)], model_week.values, label=label,
↳**line_kwargs(label))
ax.set_xlabel("Week of 2023")
ax.set_ylabel("GWh")
ax.set_title("Weekly scarcity proxy: OCGT + load shedding")
ax.legend(loc="upper right", frameon=False)
plt.tight_layout()
plt.show()

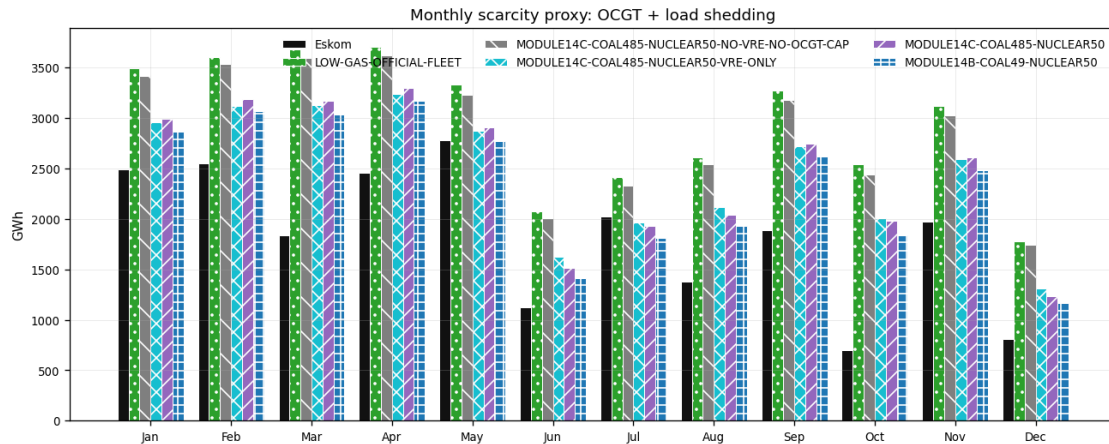
```



1.4.3 3.3 Monthly scarcity bar chart

```
[156]: reference = scarcity_series(next(iter(series.values()))["eskom"] if series else
↳ eskom_timeseries(eskom_raw.index)).resample("ME").sum() / 1e3
monthly_scarcity = pd.DataFrame({"Eskom": reference.values}, index=MONTH_LABELS)
for solve, label in SOLVE_LABELS.items():
    if solve in series:
        monthly_scarcity[label] = (scarcity_series(series[solve]["model"]).
↳ resample("ME").sum() / 1e3).values

fig, ax = plt.subplots(figsize=(11, 4.5))
x = np.arange(len(monthly_scarcity.index))
width = min(0.15, 0.82 / max(len(monthly_scarcity.columns), 1))
for i, column in enumerate(monthly_scarcity.columns):
    ax.bar(
        x + (i - (len(monthly_scarcity.columns) - 1) / 2) * width,
        monthly_scarcity[column],
        width,
        label=column,
        color=SERIES_STYLE[column]["color"],
        hatch=SERIES_STYLE[column]["hatch"],
        edgecolor="white",
        linewidth=0.6,
    )
ax.set_xticks(x)
ax.set_xticklabels(monthly_scarcity.index)
ax.set_ylabel("GWh")
ax.set_title("Monthly scarcity proxy: OCGT + load shedding")
ax.legend(loc="upper right", ncol=3, frameon=False)
plt.tight_layout()
plt.show()
```



1.5 4. Capacity factor summary

Compare annual utilisation against installed capacity where capacity-factor interpretation is meaningful. Model capacity factors use network nameplate values, while Eskom reference values use available capacity anchors.

```
[157]: CF_CARRIERS = ["coal", "nuclear", "ocgt", "sasol_coal", "sasol_gas", "hydro",
    ↪ "wind", "solar_pv", "csp"]
MODEL_CAPACITY_CARRIERS = {
    "coal": ("generators", ["coal"]),
    "nuclear": ("generators", ["nuclear"]),
    "ocgt": ("generators", ["OCGT", "ocgt", "ocgt_diesel", "ocgt_gas"]),
    "sasol_coal": ("generators", ["sasol_coal"]),
    "sasol_gas": ("generators", ["sasol_gas"]),
    "hydro": ("storage_units", ["hydro"]),
    "wind": ("generators", ["onwind", "wind"]),
    "solar_pv": ("generators", ["solar", "solar_pv"]),
    "csp": ("generators", ["csp", "solar_csp"]),
}
ESKOM_CAPACITY_MAP = {
    "coal": "Installed Eskom Capacity",
    "nuclear": "Installed Eskom Capacity",
    "ocgt": "Installed Eskom Capacity",
    "hydro": "Installed Eskom Capacity",
    "wind": "Wind Installed Capacity",
    "solar_pv": "PV Installed Capacity",
    "csp": "CSP Installed Capacity",
}

def model_capacity_mw(net, carrier):
    component, carriers = MODEL_CAPACITY_CARRIERS[carrier]
```

```

if component == "generators":
    subset = net.generators[net.generators.carrier.isin(carriers)]
    return float(subset.p_nom.sum()) if "p_nom" in subset else np.nan
subset = net.storage_units[net.storage_units.carrier.isin(carriers)]
cap = float(subset.p_nom.sum()) if "p_nom" in subset and len(subset) else 0.
↪0
if cap > 0:
    return cap
# Fallback: RSA models hydro as generators rather than storage_units
gen_subset = net.generators[net.generators.carrier.isin(carriers + ↪
↪["hydro", "hydro_import"])]
return float(gen_subset.p_nom.sum()) if "p_nom" in gen_subset and ↪
↪len(gen_subset) else np.nan

def eskom_capacity_mw(carrier):
    if eskom_capacity_anchors.empty():
        return np.nan
    rows = eskom_capacity_anchors[eskom_capacity_anchors["source_column"].
↪eq(ESKOM_CAPACITY_MAP.get(carrier, ""))]
    if rows.empty:
        return np.nan
    for col in ["p_nom_mw_2023_max", "p_nom_mw_2023_mean", "p_nom_mw_2023_min"]:
        vals = pd.to_numeric(rows[col], errors="coerce").dropna()
        if not vals.empty:
            return float(vals.max())
    return np.nan

cf_rows = []
reference = next(iter(series.values()))["eskom"] if series else ↪
↪eskom_timeseries(eskom_raw.index)
for carrier in CF_CARRIERS:
    row = {"carrier": carrier}
    cap = eskom_capacity_mw(carrier)
    row["Eskom CF"] = annual_gwh(reference[carrier]) / (cap * 8760 / 1000) if ↪
↪cap and np.isfinite(cap) and cap > 0 else np.nan
    for solve, label in SOLVE_LABELS.items():
        if solve in networks and solve in series:
            cap_model = model_capacity_mw(networks[solve], carrier)
            row[f"{label} CF"] = annual_gwh(series[solve]["model"][carrier]) / ↪
↪(cap_model * 8760 / 1000) if cap_model and np.isfinite(cap_model) and ↪
↪cap_model > 0 else np.nan
        else:
            row[f"{label} CF"] = np.nan
    cf_rows.append(row)

capacity_factor_summary = pd.DataFrame(cf_rows).set_index("carrier")

```

```
capacity_factor_summary.style.format("{:.1%}", na_rep="").
    ↪background_gradient(cmap="YlGnBu")
```

[157]: <pandas.io.formats.style.Styler at 0x322f7d690>

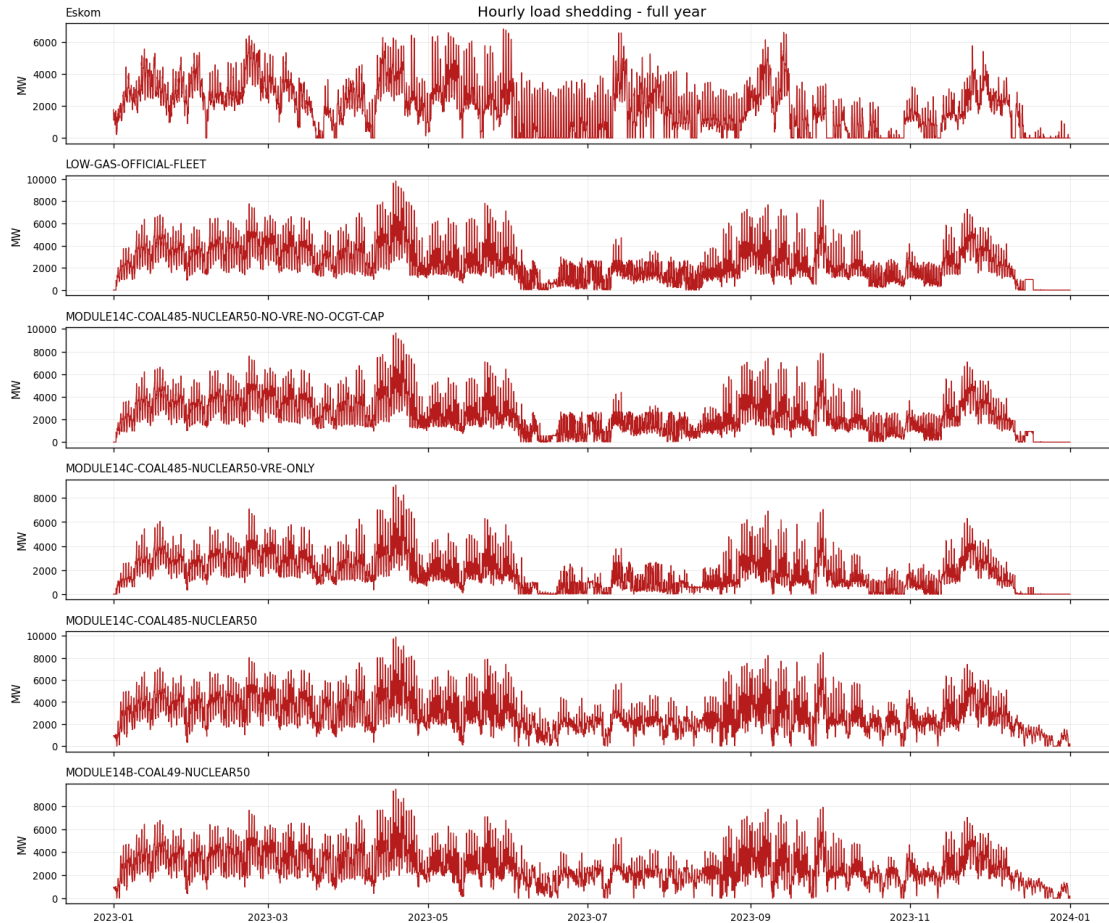
1.6 5. Load shedding deep dive

Inspect the magnitude, seasonality, distribution, and duration of unmet-energy events. This section treats load shedding as a scarcity signal, not as physical generation.

1.6.1 5.1 Hourly load shedding - full year

```
[158]: panels = [("Eskom", next(iter(series.values()))["eskom"]["load_shedding"] if
    ↪series else eskom_timeseries(eskom_raw.index)["load_shedding"])]
for solve, label in SOLVE_LABELS.items():
    if solve in series:
        panels.append((label, series[solve]["model"]["load_shedding"]))

fig, axes = plt.subplots(len(panels), 1, figsize=(13, 1.8 * len(panels)),
    ↪sharex=True)
if len(panels) == 1:
    axes = [axes]
for ax, (label, values) in zip(axes, panels):
    ax.plot(values.index, values.values, color=CARRIER_COLORS["load_shedding"],
    ↪lw=0.8)
    ax.set_ylabel("MW")
    ax.set_title(label, loc="left", fontsize=9)
axes[0].set_title("Hourly load shedding - full year", loc="center", fontsize=12)
plt.tight_layout()
plt.show()
```



1.6.2 5.2 Load shedding seasonality - monthly GWh

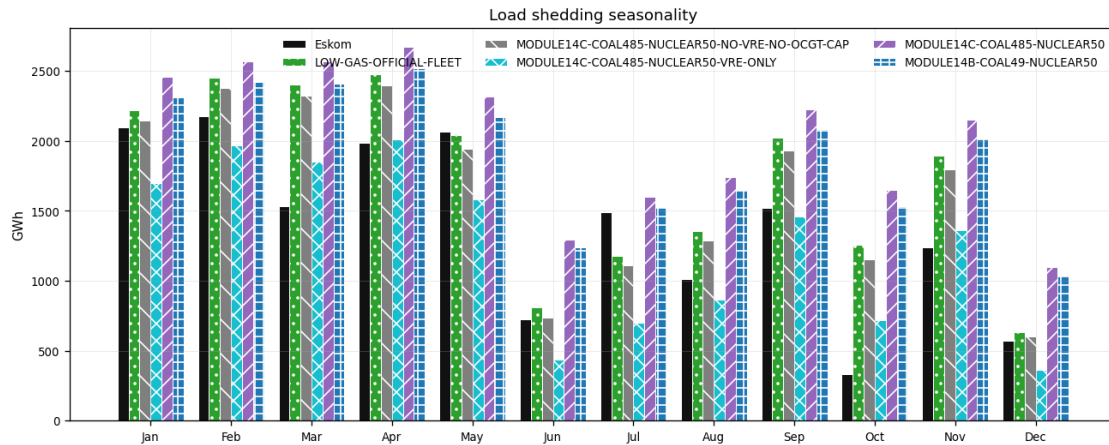
```
[159]: monthly_shed = pd.DataFrame(
    {"Eskom": monthly_gwh(next(iter(series.values()))["eskom"] if series else
    ↪eskom_timeseries(eskom_raw.index))["load_shedding"].values},
    index=MONTH_LABELS,
)
for solve, label in SOLVE_LABELS.items():
    if solve in series:
        monthly_shed[label] =
    ↪monthly_gwh(series[solve]["model"])["load_shedding"].values

fig, ax = plt.subplots(figsize=(11, 4.5))
x = np.arange(len(monthly_shed.index))
width = min(0.15, 0.82 / max(len(monthly_shed.columns), 1))
for i, column in enumerate(monthly_shed.columns):
```

```

    ax.bar(x + (i - (len(monthly_shed.columns) - 1) / 2) * width,
    ↪monthly_shed[column], width, label=column,
    ↪color=SERIES_STYLE[column]["color"], hatch=SERIES_STYLE[column]["hatch"],
    ↪edgecolor="white", linewidth=0.6)
ax.set_xticks(x)
ax.set_xticklabels(monthly_shed.index)
ax.set_ylabel("GWh")
ax.set_title("Load shedding seasonality")
ax.legend(loc="upper right", ncol=3, frameon=False)
plt.tight_layout()
plt.show()

```



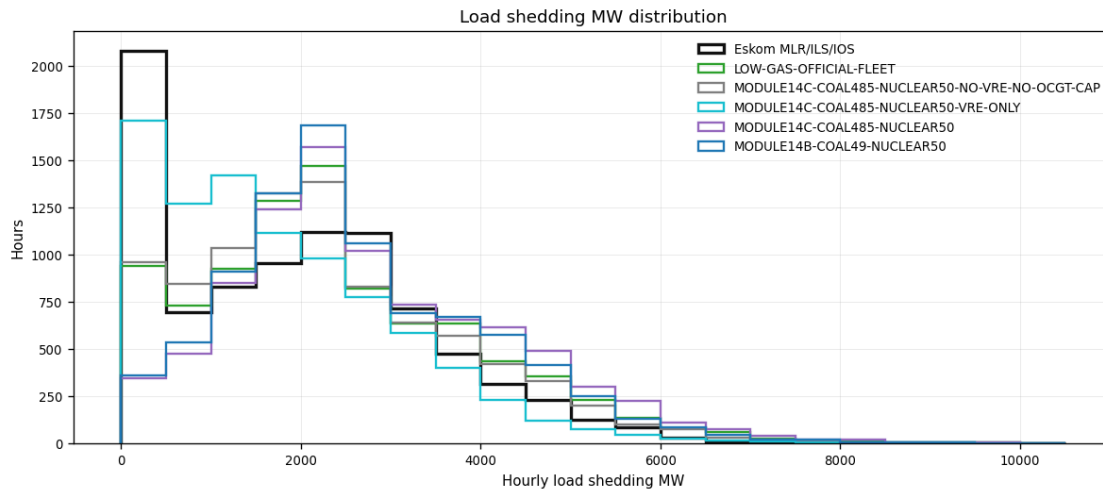
1.6.3 5.3 Stage-proxy histogram

```

[160]: reference = next(iter(series.values()))["eskom"]["load_shedding"] if series
    ↪else eskom_timeseries(eskom_raw.index)["load_shedding"]
max_value = max([reference.max()] + [series[s]["model"]["load_shedding"].max()
    ↪for s in series])
bins = np.arange(0, max_value + 1000, 500)
fig, ax = plt.subplots(figsize=(10, 4.5))
ax.hist(reference.values, bins=bins, histtype="step", lw=2.0,
    ↪color=SERIES_STYLE["Eskom"]["color"], label="Eskom MLR/ILS/IOS")
for solve, label in SOLVE_LABELS.items():
    if solve in series:
        ax.hist(series[solve]["model"]["load_shedding"].values, bins=bins,
    ↪histtype="step", lw=1.4, color=SERIES_STYLE[label]["color"], label=label)
ax.set_xlabel("Hourly load shedding MW")
ax.set_ylabel("Hours")
ax.set_title("Load shedding MW distribution")
ax.legend(loc="upper right", frameon=False)
plt.tight_layout()

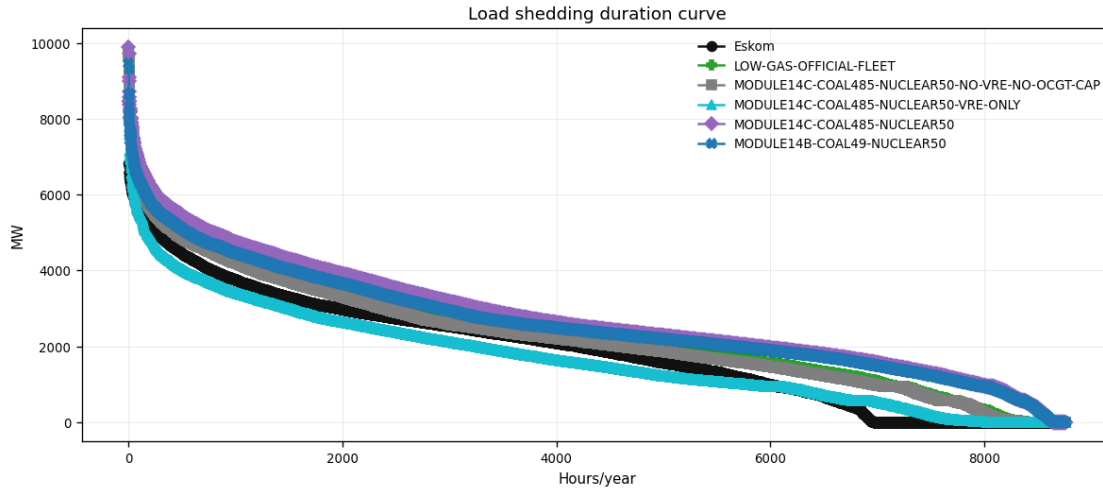
```

```
plt.show()
```



1.6.4 5.4 Duration curve

```
[161]: fig, ax = plt.subplots(figsize=(10, 4.5))
reference = next(iter(series.values()))["eskom"]["load_shedding"] if series_
    else eskom_timeseries(eskom_raw.index)["load_shedding"]
ax.plot(np.sort(reference.values)[::-1], label="Eskom", **line_kwargs("Eskom"))
for solve, label in SOLVE_LABELS.items():
    if solve in series:
        ax.plot(np.sort(series[solve]["model"]["load_shedding"].values)[::-1],
            label=label, **line_kwargs(label))
ax.set_xlabel("Hours/year")
ax.set_ylabel("MW")
ax.set_title("Load shedding duration curve")
ax.legend(loc="upper right", frameon=False)
plt.tight_layout()
plt.show()
```

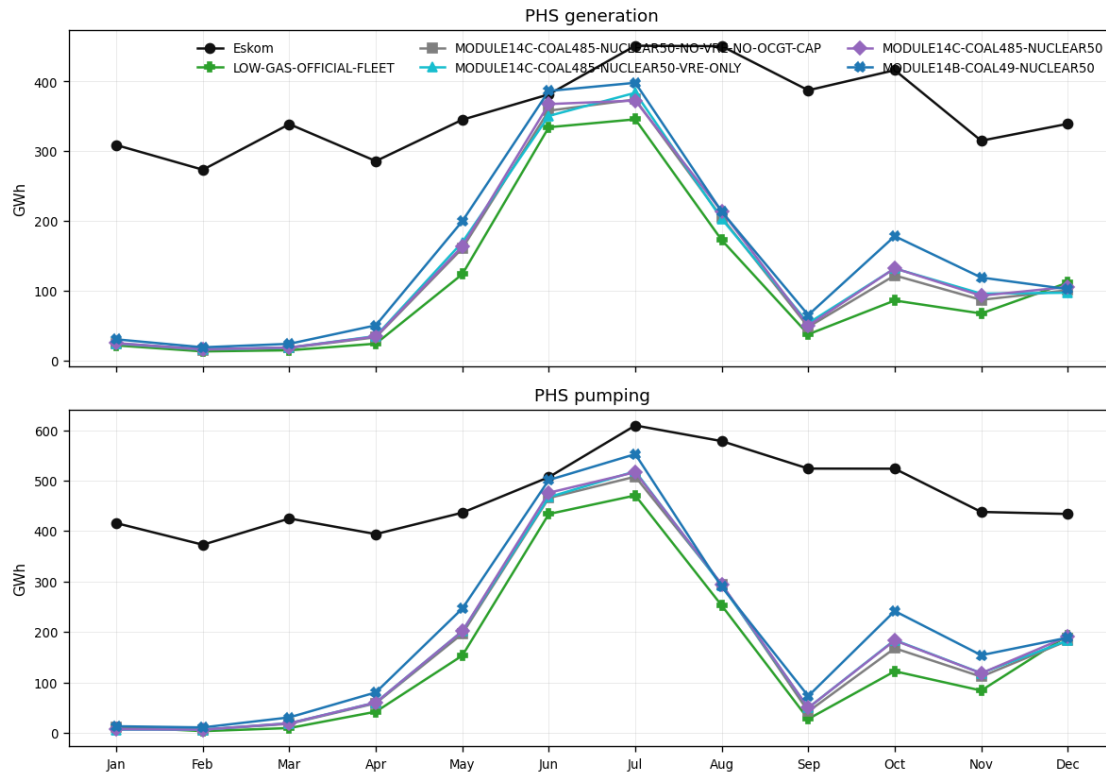


1.7 6. PHS deep dive

Isolate pumped hydro because it remains one of the largest calibration gaps. The diagnostics compare generation, pumping, net monthly balance, and a round-trip efficiency proxy.

1.7.1 6.1 Monthly PHS generation and pumping

```
[162]: reference = next(iter(series.values()))["eskom"] if series else _
        eskom_timeseries(eskom_raw.index)
fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
for carrier, ax, title in [
    ("phs_generation", axes[0], "PHS generation"),
    ("phs_pumping", axes[1], "PHS pumping"),
]:
    ax.plot(MONTHS, monthly_gwh(reference)[carrier].values, label="Eskom", _
        **line_kwargs("Eskom"))
    for solve, label in SOLVE_LABELS.items():
        if solve in series:
            ax.plot(MONTHS, monthly_gwh(series[solve]["model"])[carrier].
                values, label=label, **line_kwargs(label))
    ax.set_xticks(MONTHS)
    ax.set_xticklabels(MONTH_LABELS)
    ax.set_ylabel("GWh")
    ax.set_title(title)
axes[0].legend(loc="upper right", ncol=3, frameon=False)
plt.tight_layout()
plt.show()
```



1.7.2 6.2 PHS net energy by month

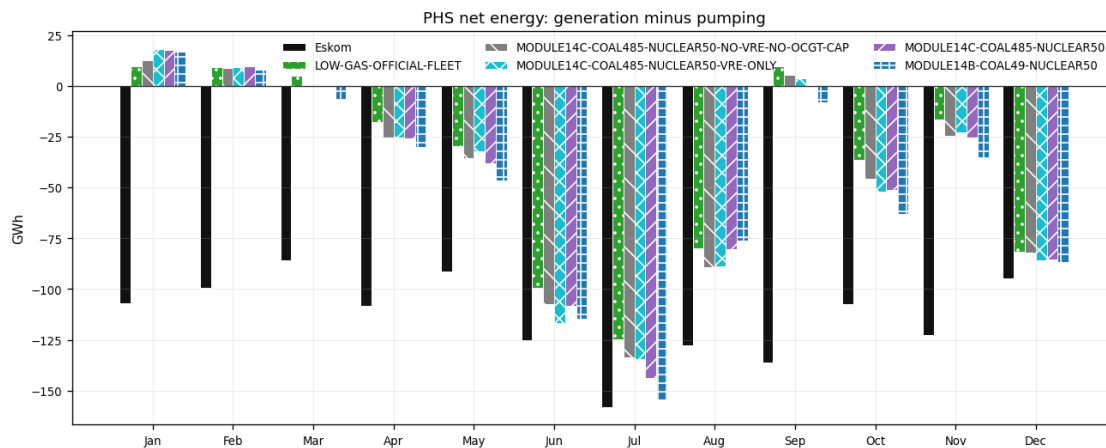
```
[163]: reference = next(iter(series.values()))["eskom"] if series else _
        eskom_timeseries(eskom_raw.index)
phs_net = pd.DataFrame(
    {"Eskom": (monthly_gwh(reference)["phs_generation"] -
        monthly_gwh(reference)["phs_pumping"]).values},
    index=MONTH_LABELS,
)
for solve, label in SOLVE_LABELS.items():
    if solve in series:
        m = monthly_gwh(series[solve]["model"])
        phs_net[label] = (m["phs_generation"] - m["phs_pumping"]).values

fig, ax = plt.subplots(figsize=(11, 4.5))
x = np.arange(len(phs_net.index))
width = min(0.15, 0.82 / max(len(phs_net.columns), 1))
for i, column in enumerate(phs_net.columns):
    ax.bar(x + (i - (len(phs_net.columns) - 1) / 2) * width, phs_net[column],
        width, label=column, color=SERIES_STYLE[column]["color"],
        hatch=SERIES_STYLE[column]["hatch"], edgecolor="white", linewidth=0.6)
ax.axhline(0, color="#333333", lw=0.8)
```

```

ax.set_xticks(x)
ax.set_xticklabels(phs_net.index)
ax.set_ylabel("GWh")
ax.set_title("PHS net energy: generation minus pumping")
ax.legend(loc="upper right", ncol=3, frameon=False)
plt.tight_layout()
plt.show()

```



1.7.3 6.3 PHS round-trip efficiency proxy

```

[164]: reference = next(iter(series.values()))["eskom"] if series else
↳eskom_timeseries(eskom_raw.index)
rows = []
for label, frame in [("Eskom", reference)] + [(SOLVE_LABELS[s],
↳series[s]["model"]) for s in SOLVE_LABELS if s in series]:
    generation = annual_gwh(frame["phs_generation"])
    pumping = annual_gwh(frame["phs_pumping"])
    rows.append({
        "Solve": label,
        "PHS generation GWh": generation,
        "PHS pumping GWh": pumping,
        "Round-trip proxy": generation / pumping if pumping > 0 else np.nan,
        "Note": "Eskom expected range about 0.75-0.80" if label == "Eskom" else
↳"",
    })
phs_efficiency = pd.DataFrame(rows)
phs_efficiency.style.format({
    "PHS generation GWh": "{:,.0f}",
    "PHS pumping GWh": "{:,.0f}",
    "Round-trip proxy": "{:,.2f}",
}, na_rep="")

```

[164]: <pandas.io.formats.style.Styler at 0x30dd322d0>

1.8 7. Summary scorecard

Provide a compact final reference table for external review. The carrier rows keep the named Eskom-column comparison used throughout the report. The accounting rows at the bottom separate that named-carrier subtotal from aggregate source columns, omitted Eskom categories, storage pumping, net imports, and the demand-balance residual. This prevents the named physical-generation subtotal from being read as a full system energy-balance total.

```
[165]: SHORT = {
    "LOW-GAS-OFFICIAL-FLEET": "LOW_OFF_CAL",
    "MODULE14C-COAL485-NUCLEAR50-NO-VRE-NO-OCGT-CAP": "M14C_BASE",
    "MODULE14C-COAL485-NUCLEAR50-VRE-ONLY": "M14C_VRE",
    "MODULE14C-COAL485-NUCLEAR50": "M14C_FULL",
    "MODULE14B-COAL49-NUCLEAR50": "M14B_C49",
}

PEARSON_SOLVES = [
    ("low_gas_official_fleet", "LOW_OFF_CAL"),
    ("module14c_coal485_nuclear50_no_vre_no_ocgt_cap", "M14C_BASE"),
    ("module14c_coal485_nuclear50_vre_only", "M14C_VRE"),
    ("module14c_coal485_nuclear50", "M14C_FULL"),
    ("module14b_coal49_nuclear50", "M14B_C49"),
]

score_rows = []
reference = next(iter(series.values()))["eskom"] if series else _
↳ eskom_timeseries(eskom_raw.index)
for carrier in CARRIER_ORDER:
    eskom_gwh = annual_gwh(reference[carrier])
    row = {"carrier": carrier, "eskom_GWh": eskom_gwh}
    for solve, label in SOLVE_LABELS.items():
        short = SHORT.get(label, label)
        if solve in series:
            model_gwh = annual_gwh(series[solve]["model"][carrier])
            row[f"{short}_GWh"] = model_gwh
            row[f"{short}_Delta_pct"] = pct_delta(model_gwh, eskom_gwh)
        else:
            row[f"{short}_GWh"] = np.nan
            row[f"{short}_Delta_pct"] = np.nan
    for solve_key, short_key in PEARSON_SOLVES:
        if solve_key in series:
            row[f"{short_key}_Pearson_r_hourly"] = _
↳ pearson(series[solve_key]["model"][carrier], reference[carrier])
            row[f"{short_key}_Pearson_r_weekly"] = _
↳ pearson(weekly_gwh(series[solve_key]["model"][carrier]), _
↳ weekly_gwh(reference[carrier]))
```

```

        else:
            row[f"{short_key}_Pearson_r_hourly"] = np.nan
            row[f"{short_key}_Pearson_r_weekly"] = np.nan
        score_rows.append(row)

all_short = [SHORT.get(label, label) for label in SOLVE_LABELS.values()]
generation_only = [r for r in score_rows if r["carrier"] in
↳ESKOM_COMPARABLE_GENERATION_CARRIERS]

def blank_accounting_row(carrier):
    row = {"carrier": carrier, "eskom_GWh": np.nan}
    for s in all_short:
        row[f"{s}_GWh"] = np.nan
        row[f"{s}_Delta_pct"] = np.nan
    for _, short_key in PEARSON_SOLVES:
        row[f"{short_key}_Pearson_r_hourly"] = np.nan
        row[f"{short_key}_Pearson_r_weekly"] = np.nan
    return row

def eskom_col_gwh(column):
    return float(eskom_raw[column].sum() / 1e3) if column in eskom_raw else 0.0

def model_load_gwh(solve):
    net = networks.get(solve)
    if net is None or net.loads_t.p_set.empty:
        return np.nan
    return float(net.loads_t.p_set.sum().sum() / 1e3)

named_total = blank_accounting_row("NAMED_COMPARABLE_GENERATION_SUBTOTAL")
named_total["eskom_GWh"] = np.nansum([r.get("eskom_GWh", 0) for r in
↳generation_only])
for s in all_short:
    named_total[f"{s}_GWh"] = np.nansum([r.get(f"{s}_GWh", 0) for r in
↳generation_only])
    named_total[f"{s}_Delta_pct"] = pct_delta(named_total[f"{s}_GWh"],
↳named_total["eskom_GWh"])

sasol_total = blank_accounting_row("MODEL_SASOL_DIAGNOSTIC_SUBTOTAL")
sasol_rows = [r for r in score_rows if r["carrier"] in
↳SASOL_DIAGNOSTIC_CARRIERS]
sasol_total["eskom_GWh"] = np.nan
for s in all_short:
    sasol_total[f"{s}_GWh"] = np.nansum([r.get(f"{s}_GWh", 0) for r in
↳sasol_rows])
    sasol_total[f"{s}_Delta_pct"] = np.nan

```

```

omitted_row =  $\square$ 
     $\hookrightarrow$ blank_accounting_row("ESKOM_OTHER_RE_AND_GAS_OMITTED_FROM_NAMED_SUBTOTAL")
omitted_row["eskom_GWh"] = eskom_col_gwh("Other RE") + eskom_col_gwh("Eskom Gas_
     $\hookrightarrow$ Generation")

unallocated_row =  $\square$ 
     $\hookrightarrow$ blank_accounting_row("ESKOM_UNALLOCATED_DISPATCHABLE_AGGREGATE_GAP")
eskom_named_dispatchable = sum(
    eskom_col_gwh(c)
    for c in [
        "Thermal Generation",
        "Nuclear Generation",
        "Eskom Gas Generation",
        "Eskom OCGT Generation",
        "Dispatchable IPP OCGT",
        "Hydro Water Generation",
        "Pumped Water Generation",
    ]
)
unallocated_row["eskom_GWh"] = eskom_col_gwh("Dispatchable Generation") -  $\square$ 
     $\hookrightarrow$ eskom_named_dispatchable

aggregate_reference =  $\square$ 
     $\hookrightarrow$ blank_accounting_row("ESKOM_AGGREGATE_GENERATION_REFERENCE")
aggregate_reference["eskom_GWh"] = eskom_col_gwh("Dispatchable Generation") +  $\square$ 
     $\hookrightarrow$ eskom_col_gwh("Total RE")
for s in all_short:
    aggregate_reference[f"{s}_GWh"] = named_total[f"{s}_GWh"]
    aggregate_reference[f"{s}_Delta_pct"] =  $\square$ 
     $\hookrightarrow$ pct_delta(aggregate_reference[f"{s}_GWh"], aggregate_reference["eskom_GWh"])

net_imports = blank_accounting_row("NET_IMPORTS_MINUS_EXPORTS")
net_imports["eskom_GWh"] = eskom_col_gwh("International Imports") -  $\square$ 
     $\hookrightarrow$ eskom_col_gwh("International Exports")
for solve, label in SOLVE_LABELS.items():
    net_imports[f"{SHORT.get(label, label)}_GWh"] = 0.0 if solve in series else  $\square$ 
     $\hookrightarrow$ np.nan

load_row = blank_accounting_row("CALIBRATED_LOAD_DEMAND")
load_row["eskom_GWh"] = annual_gwh(calibrated_demand)
for solve, label in SOLVE_LABELS.items():
    short = SHORT.get(label, label)
    load_row[f"{short}_GWh"] = model_load_gwh(solve) if solve in series else np.
     $\hookrightarrow$ nan
    load_row[f"{short}_Delta_pct"] = pct_delta(load_row[f"{short}_GWh"],  $\square$ 
     $\hookrightarrow$ load_row["eskom_GWh"])

```

```

balance_row = blank_accounting_row("BALANCE_RESIDUAL_DEMAND_MINUS_SUPPLY")
eskom_balance_supply = (
    named_total["eskom_GWh"]
    + omitted_row["eskom_GWh"]
    + net_imports["eskom_GWh"]
    + next(r["eskom_GWh"] for r in score_rows if r["carrier"] == "load_shedding")
    - next(r["eskom_GWh"] for r in score_rows if r["carrier"] == "phs_pumping")
)
balance_row["eskom_GWh"] = load_row["eskom_GWh"] - eskom_balance_supply
for solve, label in SOLVE_LABELS.items():
    short = SHORT.get(label, label)
    if solve in series:
        model_supply = (
            named_total[f"{short}_GWh"]
            + next(r[f"{short}_GWh"] for r in score_rows if r["carrier"] == "load_shedding")
            - next(r[f"{short}_GWh"] for r in score_rows if r["carrier"] == "phs_pumping")
        )
        balance_row[f"{short}_GWh"] = load_row[f"{short}_GWh"] - model_supply
        balance_row[f"{short}_Delta_pct"] = np.nan

scorecard = pd.concat(
    [
        pd.DataFrame(score_rows),
        pd.DataFrame([
            named_total,
            sasol_total,
            omitted_row,
            unallocated_row,
            aggregate_reference,
            net_imports,
            load_row,
            balance_row,
        ]),
    ],
    ignore_index=True,
)

pearson_cols = [
    f"{short}_Pearson_r_{freq}"
    for _, short in PEARSON_SOLVES
    for freq in ["hourly", "weekly"]
]
display_cols = (

```

```

    ["carrier", "eskom_GWh"]
    + [f"{s}_GWh" for s in all_short]
    + [f"{s}_Delta_pct" for s in all_short]
    + pearson_cols
)
display_cols = [c for c in display_cols if c in scorecard.columns]
scorecard_display = scorecard[display_cols]

highlight_cols = [f"{s}_Delta_pct" for s in all_short if f"{s}_Delta_pct" in
    ↪ scorecard_display.columns]
bold_cols = [c for c in scorecard_display.columns if c.
    ↪ startswith("LOW_OFF_CAL")]

def bold_highlight(data):
    styles = pd.DataFrame("", index=data.index, columns=data.columns)
    for col in bold_cols:
        if col in styles:
            styles[col] = "font-weight: bold"
    return styles

fmt = {"eskom_GWh": "{:,.0f}"}
fmt.update({f"{s}_GWh": "{:,.0f}" for s in all_short})
fmt.update({f"{s}_Delta_pct": "{:+.1f}%" for s in all_short})
fmt.update({c: "{:.3f}" for c in pearson_cols})
scorecard_display.style.format(fmt, na_rep="").
    ↪ background_gradient(cmap="RdYlGn_r", subset=highlight_cols).
    ↪ apply(bold_highlight, axis=None)

```

[165]: <pandas.io.formats.style.Styler at 0x30d7fe750>

1.9 8. Methodology note: carriers excluded from total dispatch RMSE

The total-dispatch helper used above sums Eskom-comparable physical dispatch across coal, nuclear, OCGT, wind, solar PV, CSP, hydro, and pumped-storage generation. Load shedding is handled separately as scarcity, and pumping is treated as withdrawal in the accounting rows. Biomass/run-of-river and Sasol diagnostic carriers remain outside the Eskom-comparable dispatch bucket unless a future calibrated-demand solve explicitly models them.

The Module 14c sequence is a temporary diagnostic ablation, not a clean long-run configuration state. It holds the annual coal/nuclear calibration fixed at coal EAF 0.485 and nuclear `p_max_pu` = 0.5, then toggles VRE scaling and the OCGT annual cap. This isolates which intervention improves annual carrier calibration and which intervention reallocates diesel generation into load shedding. Module 14b is retained only as a coal-49% comparator.